



OCP 2.0

Training



Introduction

- **Trainer**
 - Shawn Hurt
 - Field Applications Engineer
 - Contact information:
 - shurt@sonicsinc.com



Agenda

- **Introduction**
- **Basic Concepts : Interconnects**
- **Protocol Elements : OCP Overview**
- **OCP Protocol in Detail**
 - OCP-IP2.0 explained
 - OCP-IP2.0 vs OCP-IP1.0
 - Implementation



Open Core Protocol Introduction

The name says it all

Open



- **Non-proprietary specification controlled by an industrial consortium (special interest group)**
- **Open Core Protocol – International Partners**
 - Founded 2000 by Texas Instruments, Nokia, MIPS, UMC, Sonics
 - Current membership > 100
 - Includes Phillips, STM, Toshiba, Synopsys, Cadence, Verisity
- **Consortium administered by**
 - Same group that runs PCI SIG
- **www.ocpip.org**

Core



- **Protocol is “Core Centric”**
 - Focus is on what core needs and provides, rather than what system needs and provides
 - More in a moment...
- **The aim is to provide true CORE RE-USE**
 - Design productivity needed for the SOC age



Protocol

- **A set of rules to guide behavior for interaction or communication**
- **NOT just an interface specification**
 - Specifies a family of interfaces (tens of millions)
 - Specifies set of layered validation rules (signals, phases, transfers)
 - Provides “golden” protocol checking monitor (simulation)
 - Provides behavioral transactors (simulation)
 - Provides registration service for vendors & components
 - For system software discovery – driver loading
 - Specifies set of files, syntax, and directory structure for component description
 - Facilitates exchange of cores from providers to consumers
 - Human and machine readable (automation)

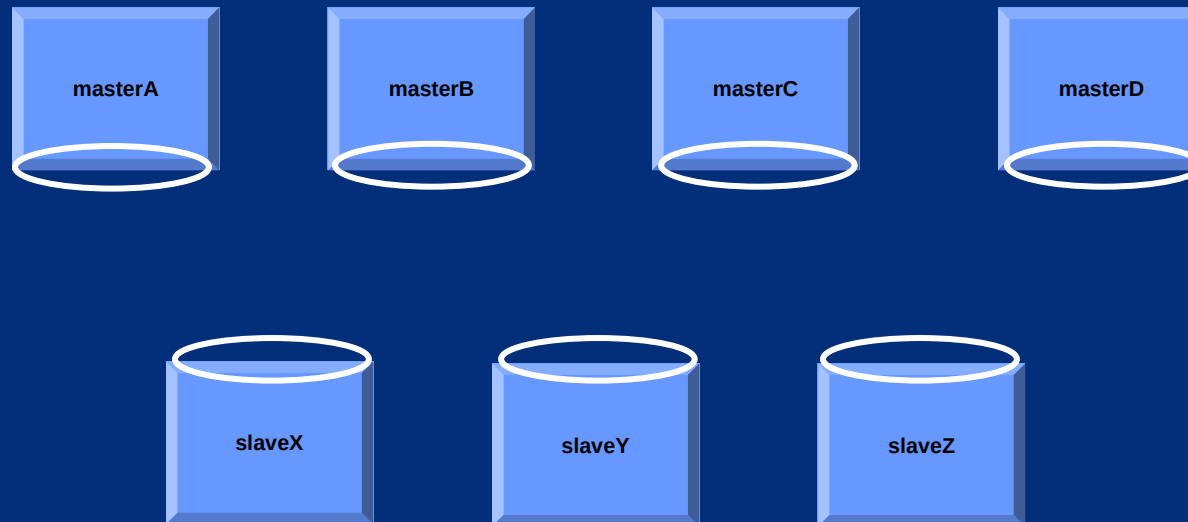


Open Core Protocol Basic Concept

A different way of thinking
...about system partitioning



“Bus Driven” System Partitioning

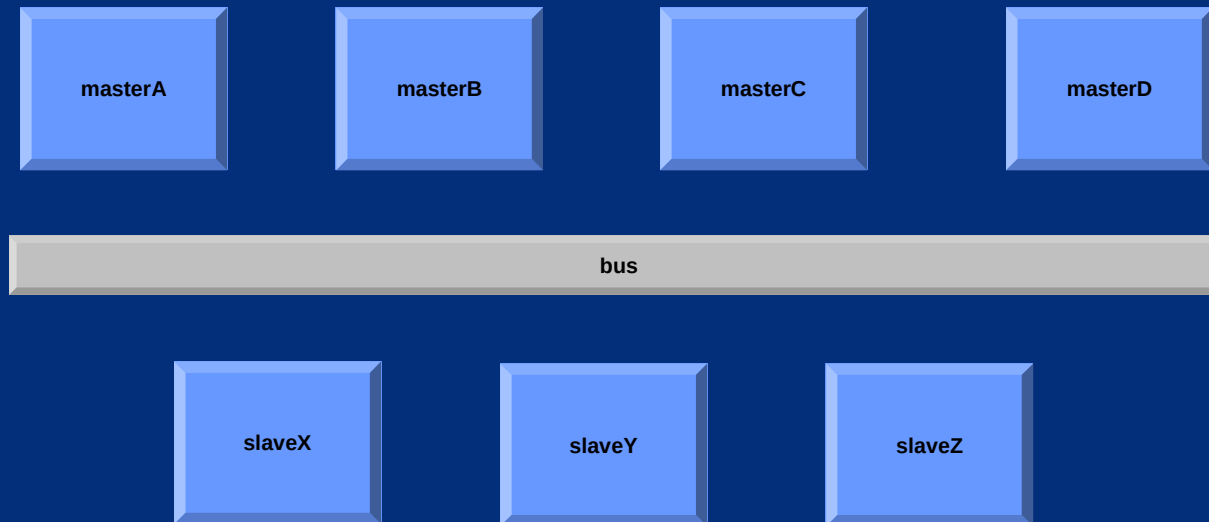


1. Start with a bunch of cores

- Note interfaces to the “system” are all different : address and data width, protocol, arbitration, interrupts and errors, etc.



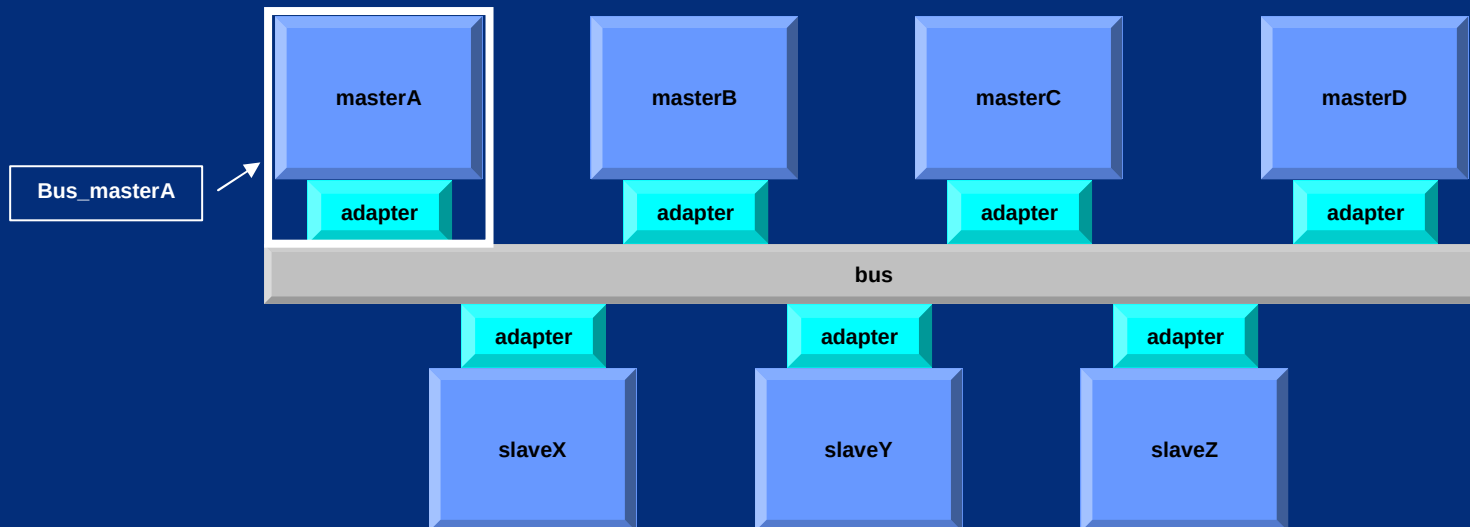
“Bus Driven” System Partitioning :2



1. **Start with a bunch of cores**
 - Note interfaces to the “system” are all different
2. **Define a Bus spec**



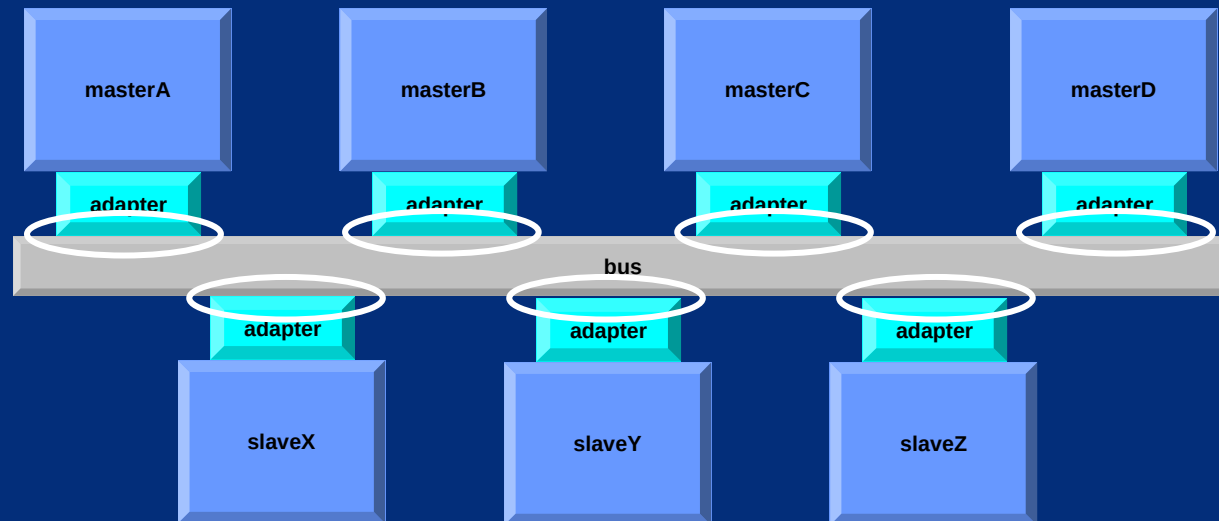
“Bus Driven” System Partitioning :3



1. **Start with a bunch of cores**
 - Note interfaces to the “system” are all different
2. **Define a Bus spec**
3. **Modify all cores to adapt to bus**
 - Then call them by that name (ahb_rom, bvci_ram, xsa_dma)
4. **Integrate all cores (& bus instance) into SOC**



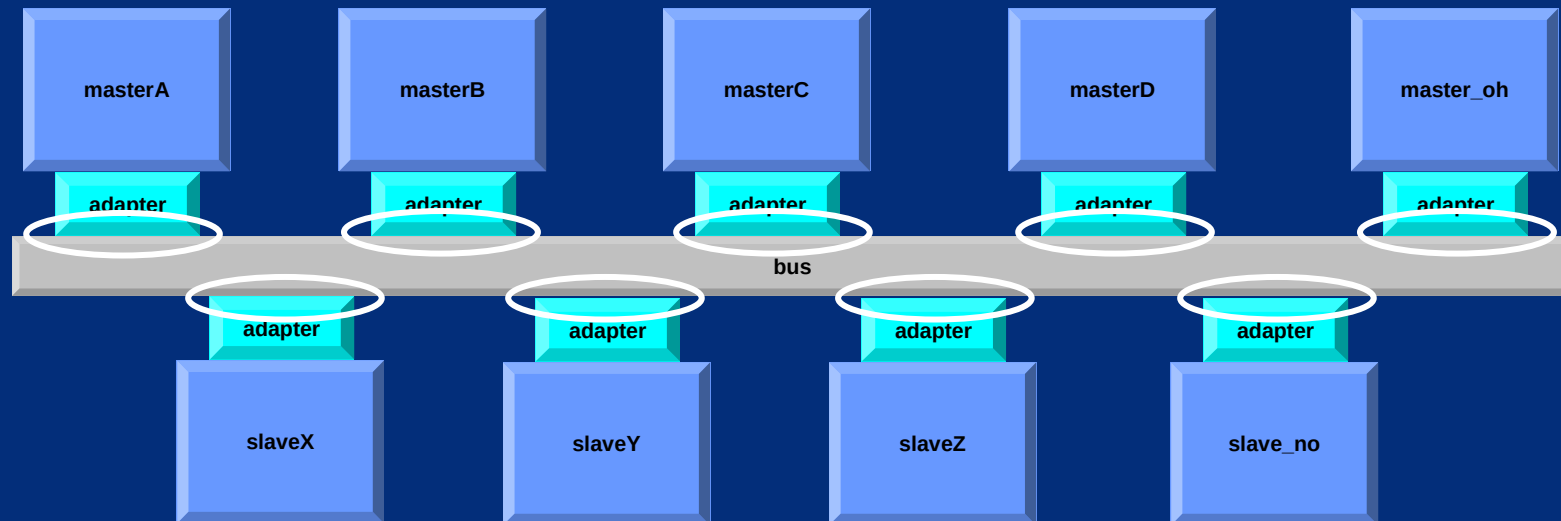
“Bus Driven” Partitioning Weakness



- **Bus interfaces are all the same**
 - Makes bus design easy
- **But core usability is now only as good as the bus**
 - Bus imposes strong constraints to the cores : bandwidth, arbitration, latency, protocol mismatch (e.g. bursts), etc.



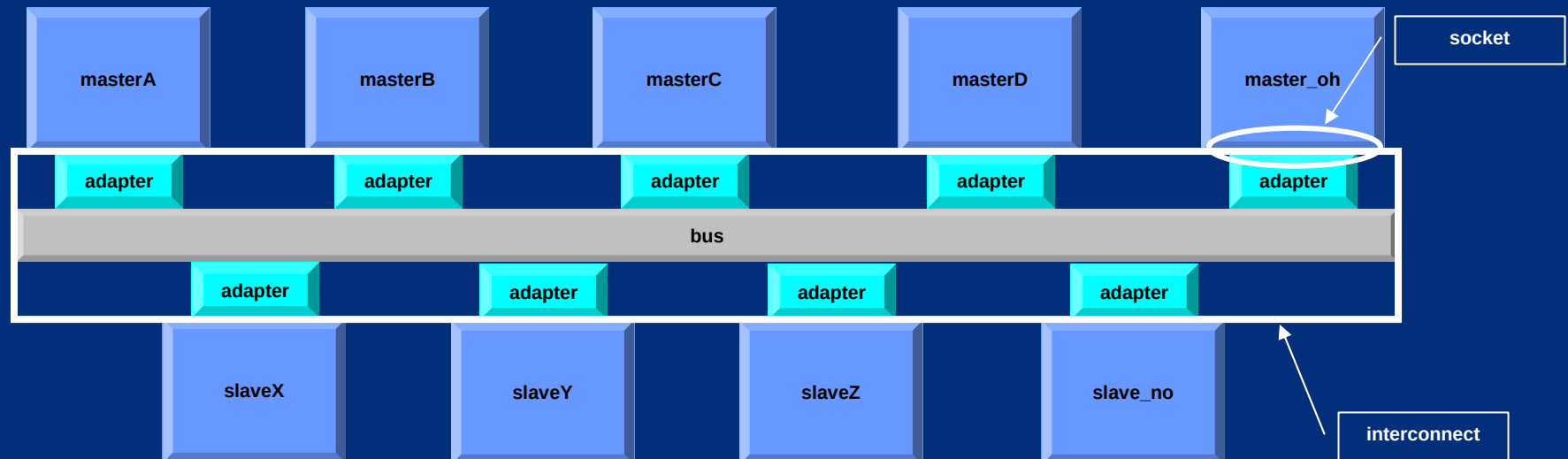
“Bus Driven” Partitioning Weakness :2



- **Add more users and oh, no !**
 - Need bus with more bandwidth
 - Change efficiency (adapter burst buffer depths, arbitration) or peak BW (data width), or both
- **All cores must be modified and re-validated !!!**
- **“Tightly coupled”**
 - System level requirements become core requirements



Socket Based System Partitioning



- **Bus and adapters are part of “interconnect module”**
- **Provides a “socket” for core to attach**
 - Sockets are NOT all the same
 - Each socket is specific to each core
- **“De-coupled”**
 - Bus (or whatever) can change, so can the bus side of adapters
 - Core side of adapter remains unchanged
 - Core requirements are independent of requirements of the system in which it is used
- **Re-Usability !!!**
- **Greatly eases test and validation**

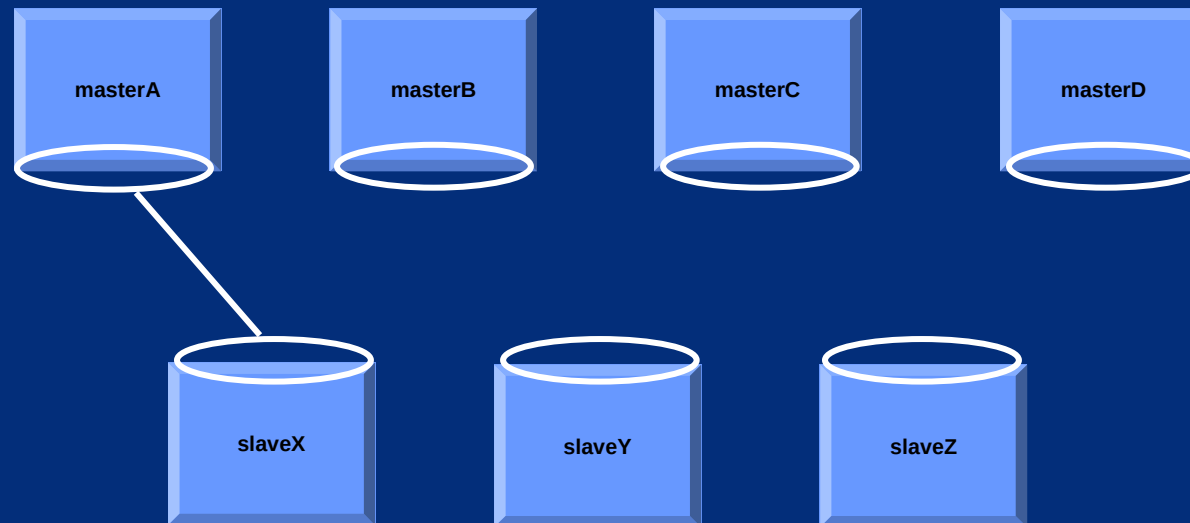


Interconnect? Some vocabulary...

- **An Interconnect is:**
 - The subsystem used to route requests in the SoC; can be build using shared links and cross bars.
 - Includes routing mechanism (address decode)
 - Includes arbiters required to share resources between initiators (access to targets and routing links)
- **A Bus or Shared link is:**
 - A time shared routing resource for requests going from initiators to targets.
- **A Cross bar is:**
 - A parallel routing resource for requests



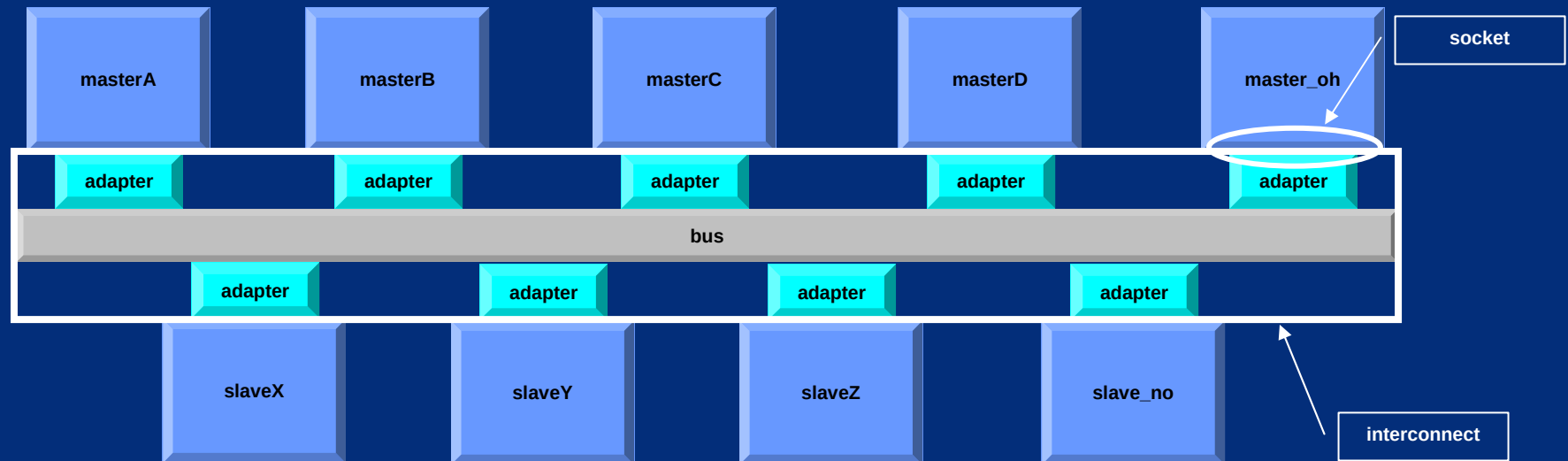
Requirements for OCP



1. **As interfaces to the “system” are all different we want a spec that:**
 - Covers a family (huge range) of interfaces (from simple RAMs to most complex CPUs...)
 - Allows to describe precisely the core’s capabilities (not limiting the core, but also not requiring more than what’s needed)
2. **Mutual compliance with the spec does NOT assure inter-operability**
 - No plug-and-play feature, too costly and useless in ASIC environment,
 - Spec must provide explicit inter-operability requirements



Requirements for OCP :2

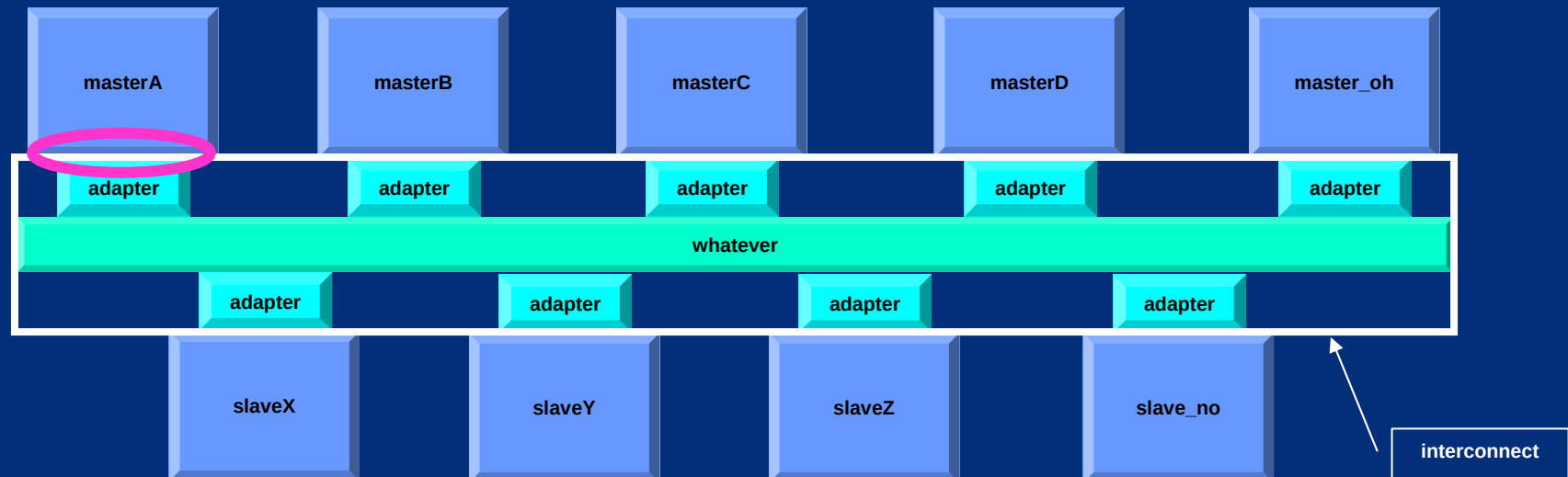


3. Each socket is specific to each core – adapter must be designed

- Spec must provide a way to formally define the core's interface
- Must include signal definitions & timing characteristics
- Spec must be VERY CLEAR about how to validate all these interfaces



Requirements for OCP :3



4. Each socket is specific to each core

- The spec shall describe a point-to-point interface protocol
- It does NOT need to deal with things associated with the system level interconnect like arbitration, address decoding, data width conversion...

5. Spec needs to deal with ALL communications between core and system - dataflow, control, test



Open Core Protocol Protocol Elements

A different way of thinking
...about IP core delivery and use

Protocol Elements: what's inside OCP?



- **A signal specification**
 - Definition of set of roles (masters, slaves) that can be played in attaching to OCP
 - Definition of set of signals (fields) that may be present in the interface, including how each role relates to them
- **A transfer specification**
 - Definition of what types of information transfers (transactions) can be performed
 - Definition of how the signals are to be used for each transaction type
- **A configuration specification**
 - Specification of what combinations of signals are permitted
- **A syntax specification**
 - Specification of file syntax for describing core and its interfaces configurations
 - Specification of file syntax for describing timing of all interface pins



Protocol Elements :2

- **A inter-operability specification**
 - Definition of set of rules for when 2 sockets can be directly connected, and how (tie-offs), and what operations they can perform.
- **A protocol checker specification**
 - Definition of what checks can and should be performed to validate that the protocol is being followed.



OCP Related Tools

- **OCP monitor**
 - Configurable monitor (read-only) module useable in simulations
 - Can attach to any OCP bundle
 - Instantiates PLI connection to TestBuilder runtime protocol checker (combines Verilog and C++).
 - Can produce trace files during simulation
- **Ocpcheck**
 - Post processing tool for OCP monitor trace files to do protocol checking
- **Ocpdis**
 - Post processing tool for OCP monitor trace files to produce human readable assembly language representation of OCP activity
- **Ocppperf**
 - Post processing tool for OCP monitor trace files to produce human readable performance analysis of OCP activity



OCP Related Tools: 2

- **Quick Models – QMaster**
 - Configurable behavioral model of an OCP master (transaction initiator)
 - More than transactor - embeds protocol execution
 - Can attach to any OCP bundle
 - Provides Verilog task based interface to control stimulus
 - Provides internal programmable execution engine to control stimulus
 - STL (Socket Transaction Language) assembly language to describe stimulus
 - Ocpasm tool to assemble STL into object code for execution engine
- **Quick Models - QSlave**
 - Configurable behavioral model of an OCP slave (transaction target)
 - More than transactor - embeds protocol execution
 - Can attach to any OCP bundle
 - Provides Verilog task based interface to control reactive behavior
 - Provides statically set parameters to control reactive behavior



OCP Related Tools: 3

- **CoreCreator**
 - GUI
 - Block diagram editor to create core testbenches
 - Design flow cockpit – launcher for the other tools
 - Wizard interface for capturing & packaging (directory structure & file syntax) core information



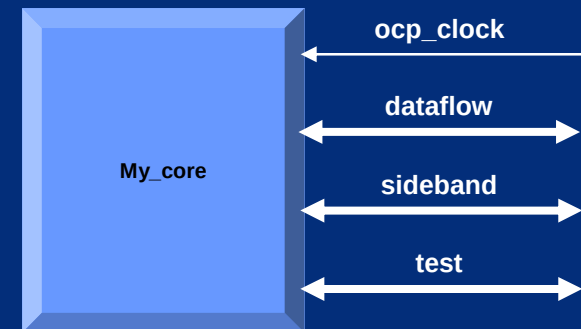
OCP-speak

- **Think of OCP as more of a language than as an interface spec**
- **It should provide the language to formally describe the interface your core already has**
 - The bus-adapted interface you currently use (“wrapping”)
 - The real core interface inside that bus-adapted version (“unwrapping”)
 - The real interface for a new core (“native”)
- **Native is best, unwrapping is a way to get to native**
- **Wrapped cores will always be limited**
 - Performance limited by the capabilities of the bus adapter that was bridged by the OCP wrapper.
 - Extra gate count (bus adapter is NOT part of native core)

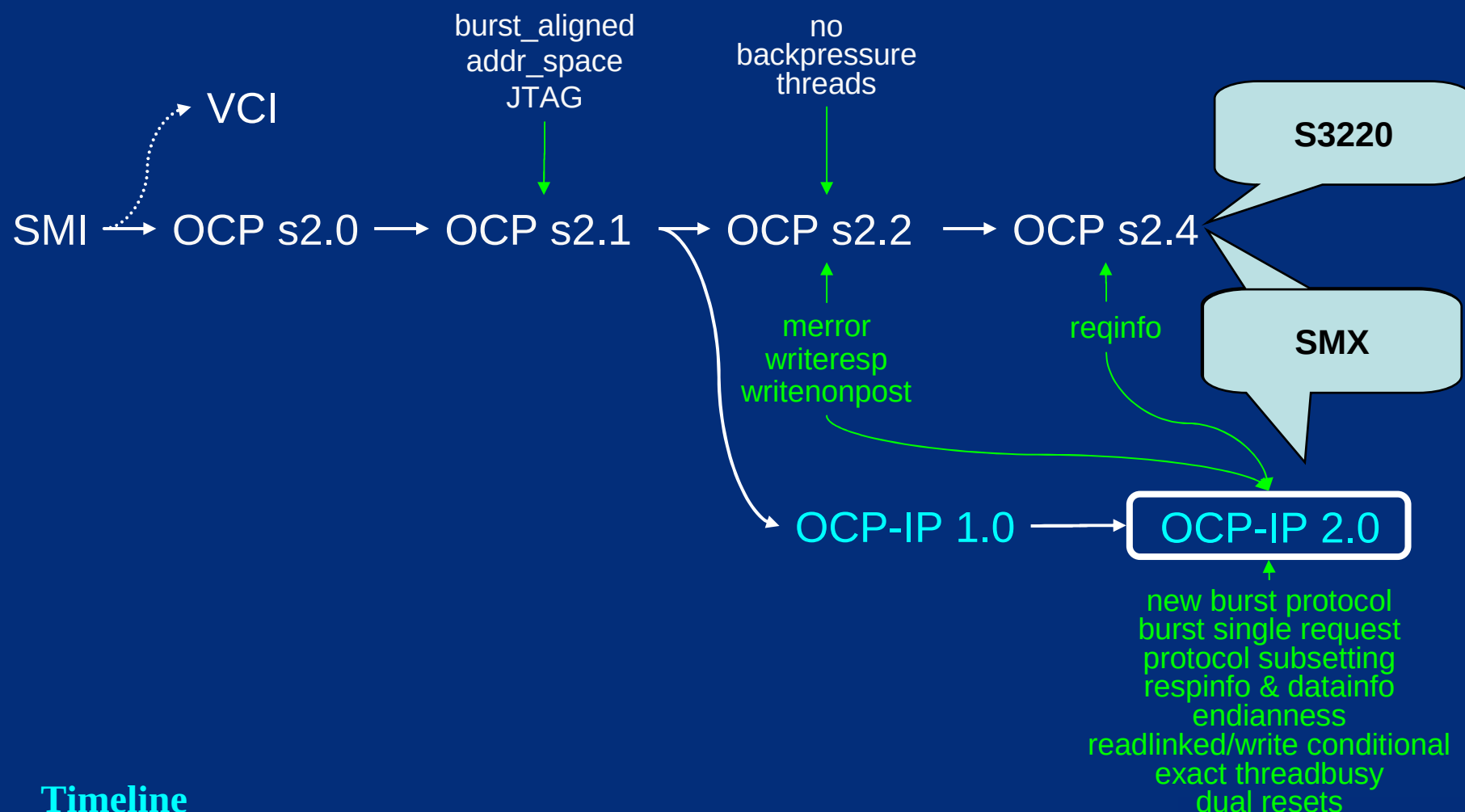


Complete, Synchronous Interface

- **3 specs in 1: separate groups of signals for**
 - Dataflow
 - Sideband (control and status)
 - Test (scan, jtag, etc)
- **Dominant topic now is Dataflow**
- **All communication is synchronous to a clock input to the core**



OCP Genealogy



Timeline

Q4 2005

Sonics Confidential: Sonics OCP 2.0 Training

27





Open Core Protocol In Detail

A different way of thinking
...about the interface you already have



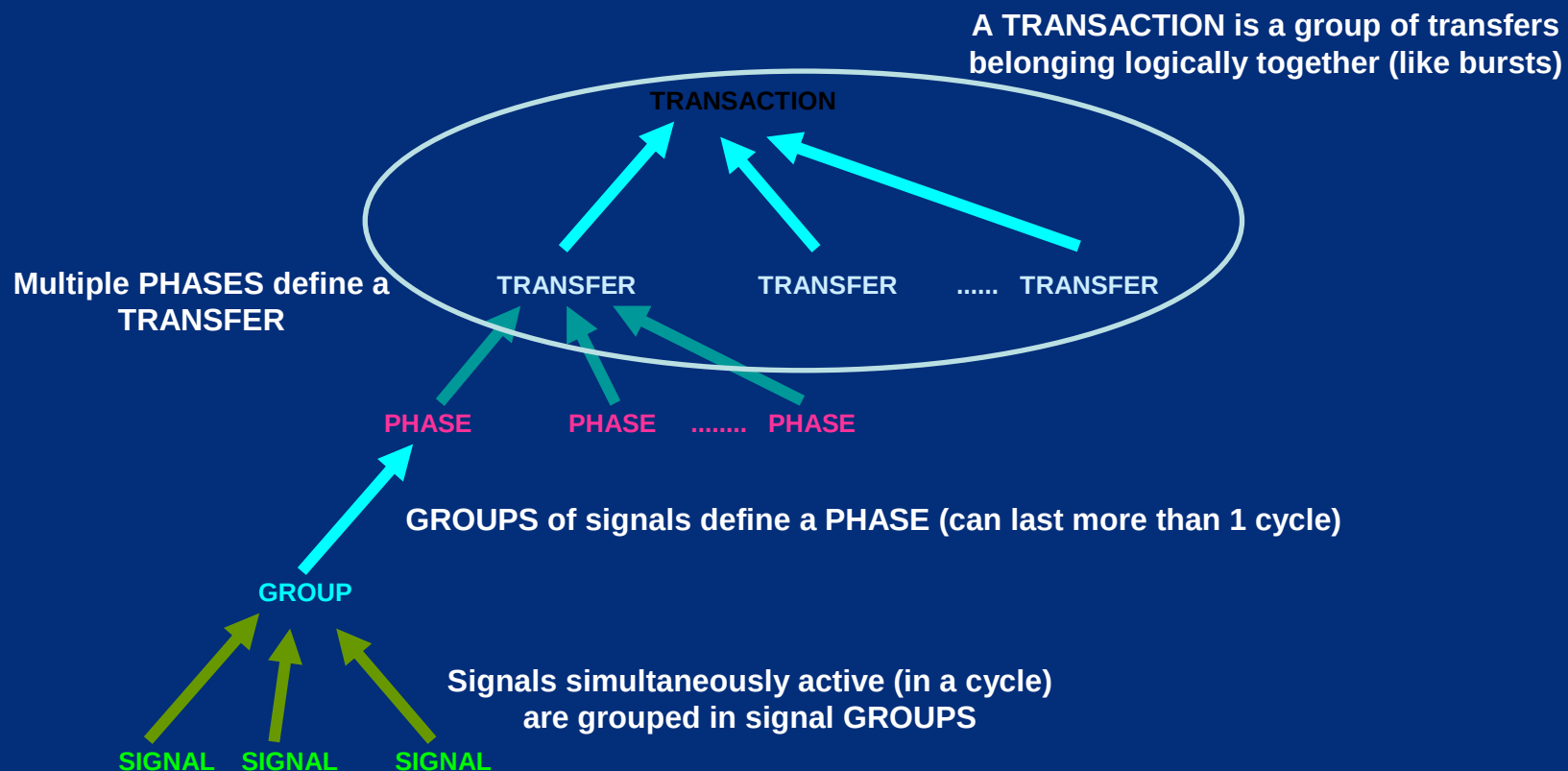
Three specs in one...

- **Dataflow:**
 - Deals with transfer of data, using commands and addresses
 - Biggest part of the protocol
- **Sideband:**
 - Deals with other, non-data related signals : reset, interrupts, flags, etc.
- **Test:**
 - OCP has provision for JTAG and scan related signals



Dataflow

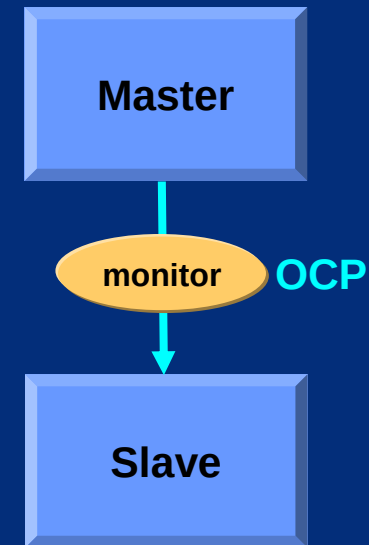
Protocol hierarchy of Dataflow Elements





Point-to-Point Transaction Interface

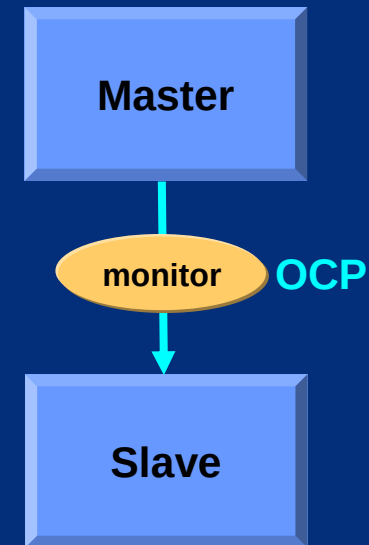
- OCP includes a set of dataflow signals that connect entities of complementary roles
- Three data dataflow roles:
 - Master
 - Initiates transactions
 - Slave
 - Services transactions
 - Monitor
 - Watches transactions
- By convention, when OCP bundle is shown (rather than it wires), an arrow shows direction of transfer initiation





OCP Transaction Types

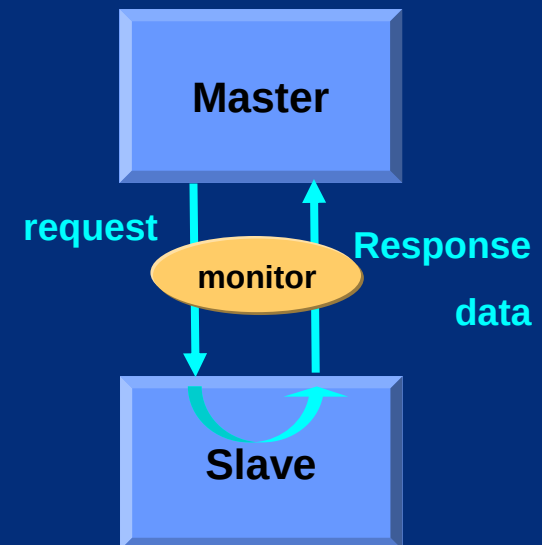
- One or more transfers build a transaction
- Bursts
 - Reads
 - Writes
 - Posted (with or without response)
 - Non-posted
 - Broadcasts
- Semaphores
 - Read-modify-write
 - Blocking
 - Non-blocking
- Configuration specifies explicitly which transactions are allowed
 - At least one type must be allowed





OCP Read Transfer

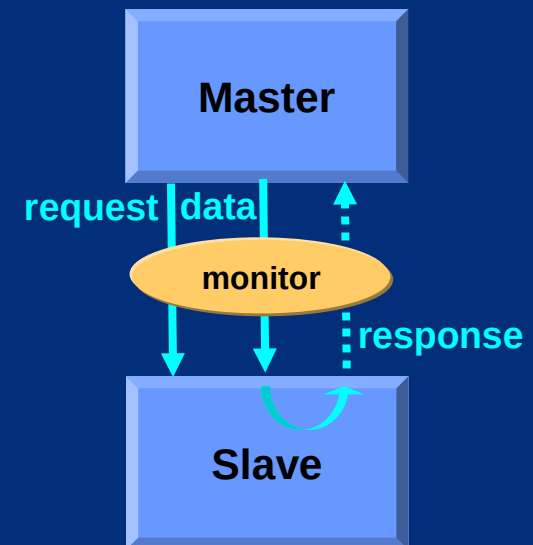
- **Master sends a request to slave**
 - Designates Read command
 - Specifies an address
- **Slave returns a response to master**
 - Indicates success or failure
 - Includes copy of the addressed data, if successful





OCP Write (Posted) Transfer

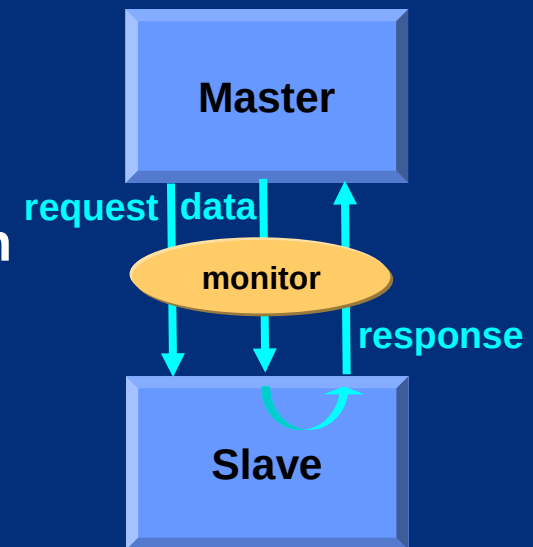
- **Master sends a request to slave**
 - Designates Write command
 - Specifies an address
- **Master sends data to slave**
- **Slave optionally returns a response to master:**
 - Only if OCP is configured for write responses
 - Indicates success speculatively
 - Subject to posted write exceptions
- **Slave modifies the addressed location with the provided data (tries to)**





OCP Non-Posted Write Transfer

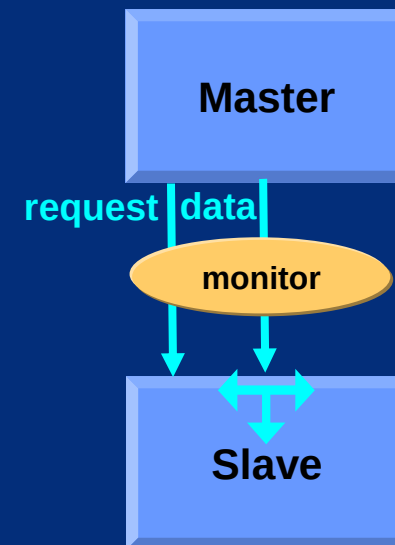
- **Master sends a request to slave**
 - Designates Non Posted Write command
 - Specifies an address
- **Master sends data to slave**
- **Slave modifies the addressed location with the provided data (tries to)**
- **Slave must return a response to master**
 - OCP must be configured for write responses
 - Fully synchronized write (response after write)





OCP Broadcast Transfer

- **Write without response with multi-point addressing**
- **Master sends a request to slave**
 - Designates Broadcast command
 - Specifies a group address
- **Master sends data to slave**
- **Slave modifies all the addressed locations with the provided data (tries to)**
- **No response to master**
 - Higher level protocols must guarantee success/progress





OCB Bursts Transaction

- All reads, writes, and broadcasts can be grouped in bursts
- Bursts have a length and an address sequence
- Bursts of length 1 (one OCB word) are legal, partial word transfer also possible
- 3 categories of bursts are supported
 - “Imprecise”
 - Speculative length
 - Multiple Request Multiple Data style
 - Early termination legal
 - “Precise”
 - Precise length
 - Multiple Request Multiple Data style
 - No early termination
 - “SRMD”
 - Single Request Multiple Data (packet) style
 - Precise length
 - No early termination
- Bursts will be detailed later



OCP Semaphore Transactions

- **Only OCP transactions with atomicity guarantees**
- **All semaphores fit read-modify-writes profile**
 - No arbitrary locked sequences
- **Indicated with special read and write commands**
- **Blocking semaphore**
 - ReadEx-Write
- **Non-blocking semaphore**
 - ReadLinked-WriteConditional



Blocking Semaphores

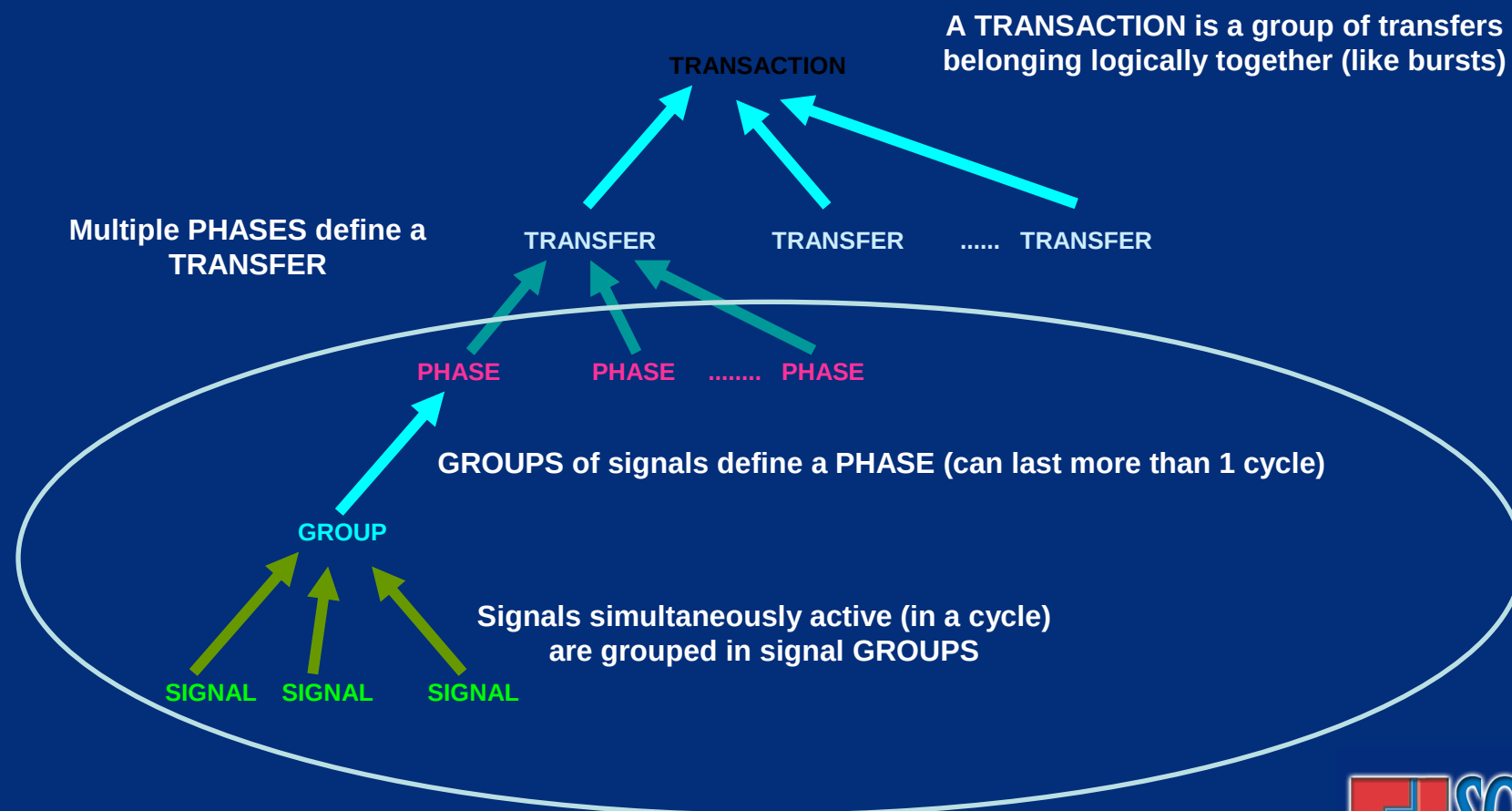
- Supported by ReadEx-Write OCP commands
- ReadEx is a read (exclusive) command
- ReadEx arbitration shall lock the addressed resource until this initiator (master and *thread* – *defined later*) unlocks it with a write
- After ReadEx only Write to same address is legal
- The write can be posted, non-posted, or broadcast
- Blocking because other masters or *threads* trying to access same slave resource will be stalled



Non-Blocking Semaphores

- Supported by ReadLinked-WriteConditional commands
- ReadLinked is a read command
- ReadLinked sets a reservation for this initiator (master and *thread*) on the addressed resource.
- For a master, after ReadLinked only WriteConditional to same address is legal
- WriteConditional only performs the write if the reservation is still in place. Any other access (from other masters) would have cleared it. WriteConditional returns success or failure response.
- Non-Blocking because other masters or *threads* are free to access same slave resource

OCP Dataflow Phases





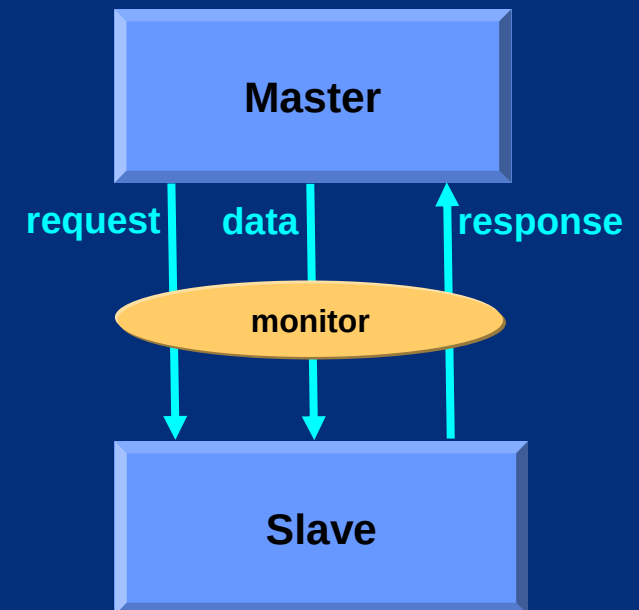
Split Transfer

- **OCP Transfers are split into phases**
 - Request phase
 - Data phase
 - Response phase
- **Transfer Pipelining**
 - Each transfer proceeds from phase to phase in a prescribed sequence
 - One transfer need not complete all phases before another transfer may begin
- **No limit set by the spec on the number of outstanding (open) transfers**
 - Permits OCP to be very high performance interface
 - Bandwidth independent of latency



Signal Groups

- OCP avoids phase interference by supporting each phase with its own independent set of signals
 - Request phase signal group
 - Data phase signal group
 - Response phase signal group
- Arrows show phase initiation direction





OCP is highly configurable

- **OCP Protocol is modular and scaleable:**
 - user can choose what to implement
- **Signal groups made up of several signal fields:**
 - Bit vectors of width $n \geq 1$, e.g. : addresses, data...
- **Many fields have a configurable size**
- **Many of the sockets fields are optional:**
 - complexity of the socket can match exactly the core needs
 - If a field has a constant value, the associated signal is not required and a default (tie-off) value can be specified:
 - Permits interfaces with minimum set of signal wires
 - Avoids test problems associated with wires that can't be toggled
- **OCP defines a standard way to describe the socket (rtl.conf syntax) at a physical (wires) and behavioral (protocol) level**



Phases Configurability

Not all phases must be implemented:

- **Request phase must always be present**
- **Data phase is optional**
 - Present if phase valid signal is present (MDataValid)
 - If phase not present, phase payload (MData) moves to request phase
- **Response phase is optional**
 - Present if phase valid signal is present (SResp)
 - If phase not present, transactions requiring response not supported
 - All read type commands
 - Non-posted writes & writes with responses



Signaling Protocol

- **When does a phase start and when is it valid?**
 - Each OCP signal group (phase) has one signal field indicating whether the phase is valid (active) or not
 - All other fields belonging to a phase including the accept signal are don't care if the phase is not valid (active)
 - No retraction : Once a phase is activated, the signal activating the phase and all other signals belonging to the same group (payload) must be held constant.





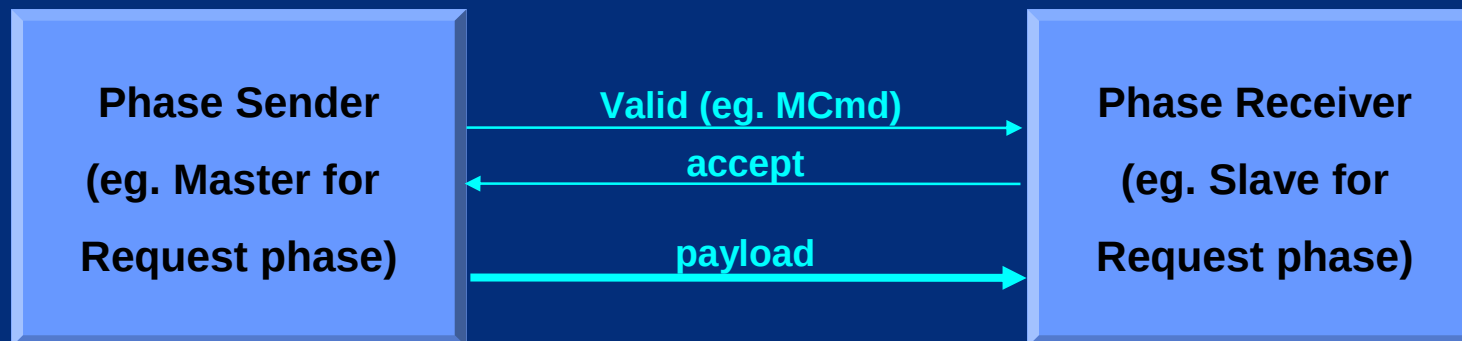
Valid Fields : MCmd, SResp and MDataValid

MCmd[2:0]	Command	SResp[1:0]	Response code
000	IDLE	00	NULL
001	WRITE	01	DVA
010	READ	10	FAIL
011	READEX	11	ERROR
100	READ-LINKED		
101	WRITE-NON-POST	MDataValid	Response code
110	WRITE-CONDITIONAL	0	Write Data not Valid
111	BROADCAST	1	Write Data Valid



Flow control Signals

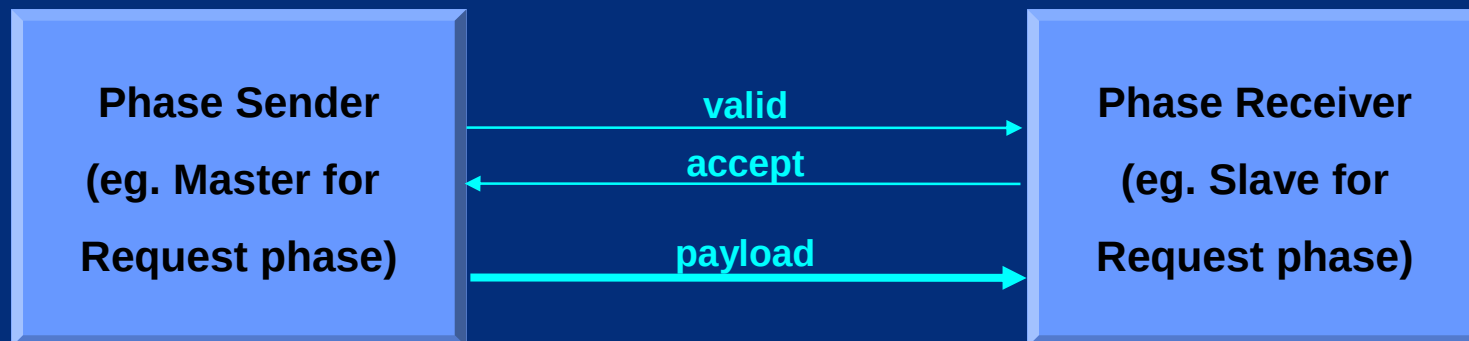
- **When does a phase end?**
 - Each phase has:
 - Handshake fields for flow control
 - Payload fields
 - Flow control fields are:
 - Valid field (multi-bit vector with 1 encoding for IDLE)
 - Accept field (single bit)





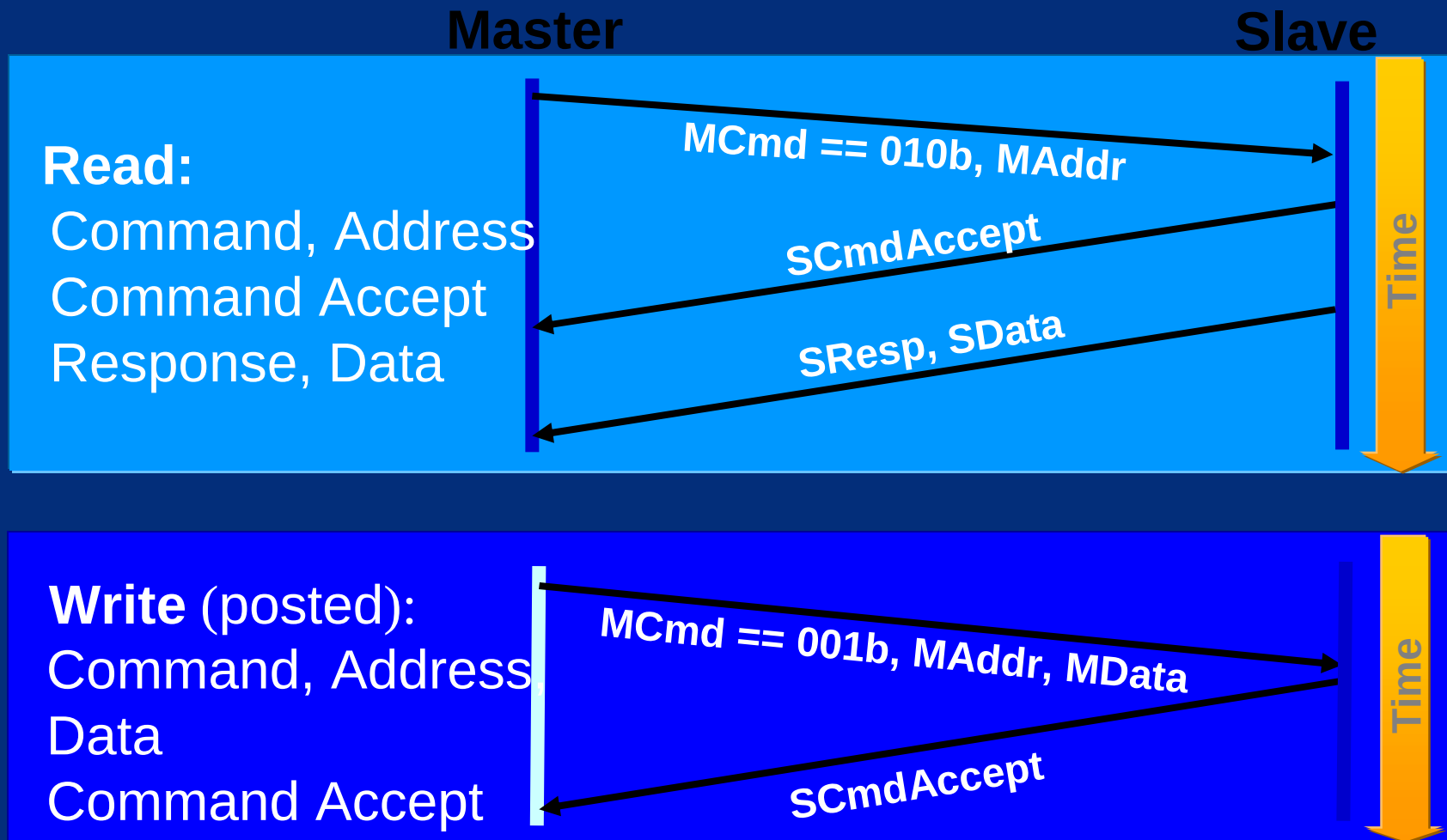
Flow control Signals (2)

Handshake field	Request phase	Data phase	Response phase
Valid	MCmd[3]	MDataValid	SResp[2]
Accept	SCmdAccept	SDataAccept	MRespAccept
Payload	Request Group Signals	Datahandshake Group Signals	Response Group Signals

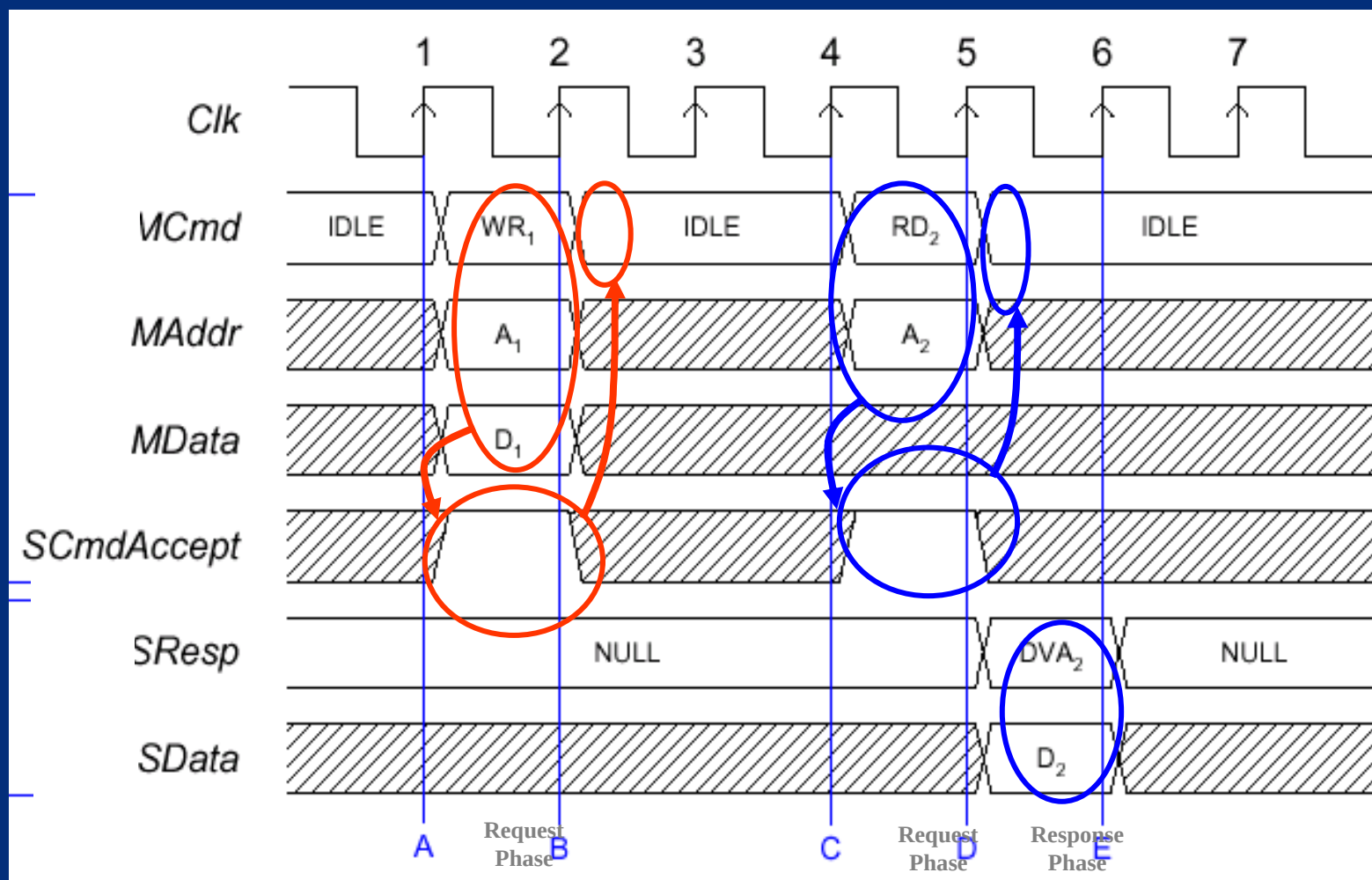




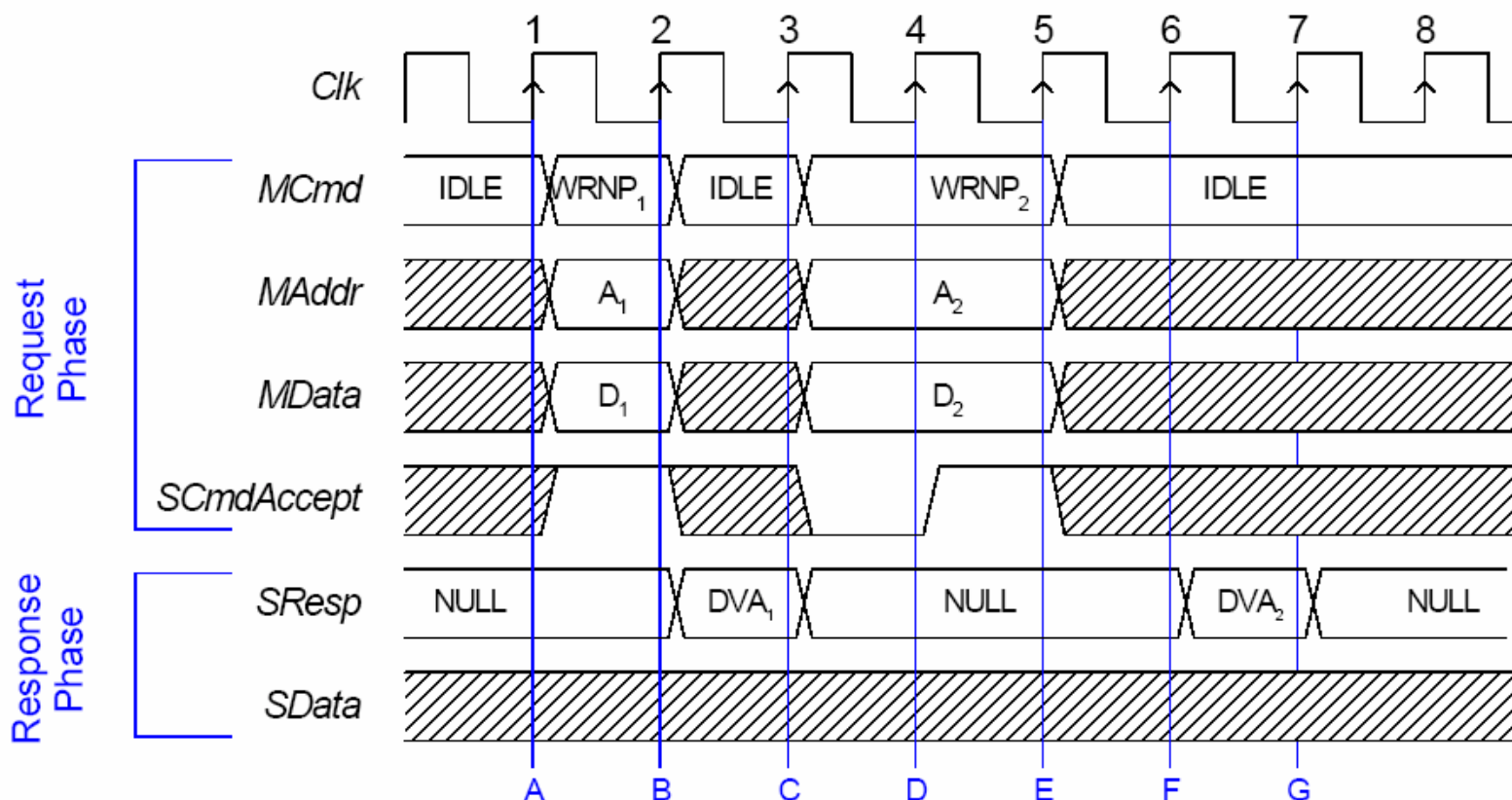
Phases of simple transfers: RD, WR



Simple Write & Read Transfer



OCP Non-Posted Write (WRNP)



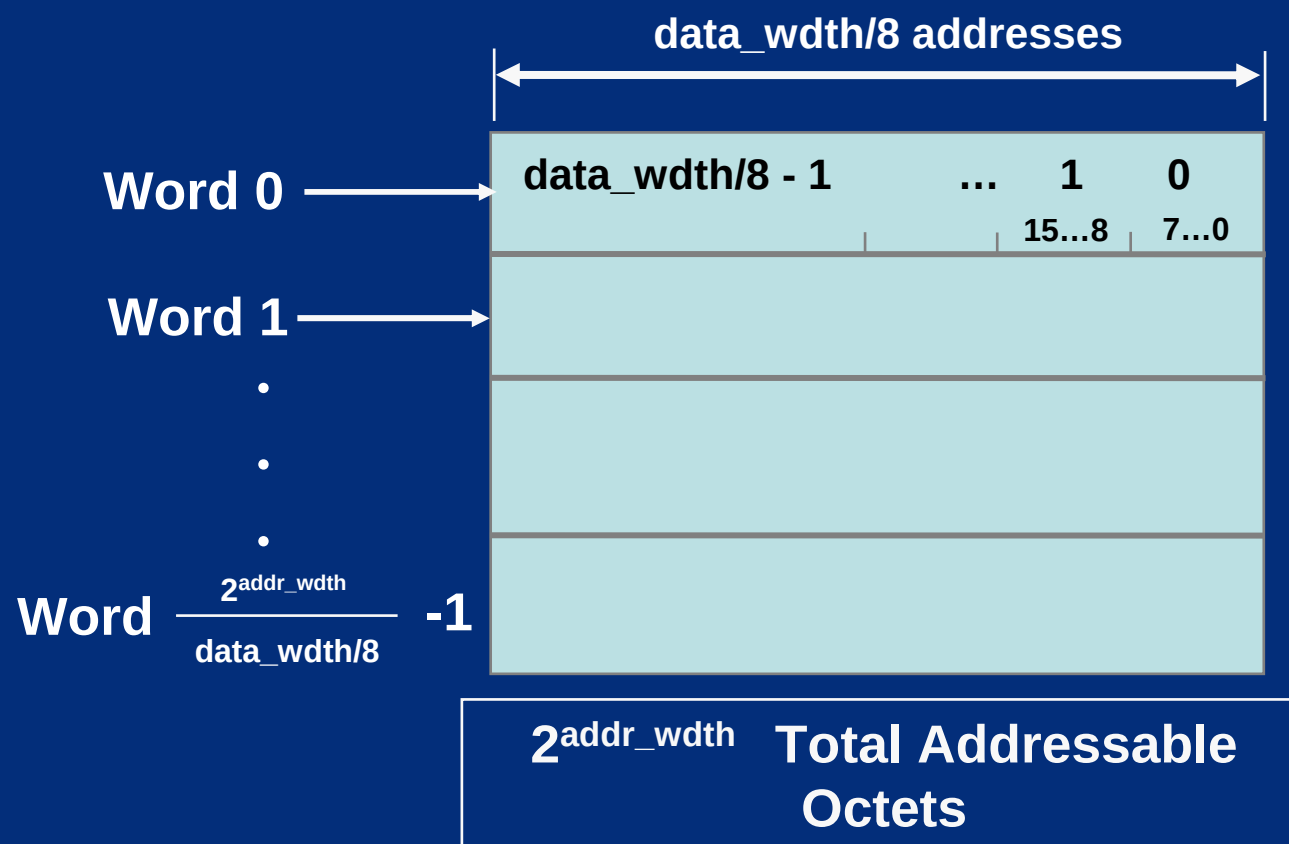


Addresses and Data in OCP world

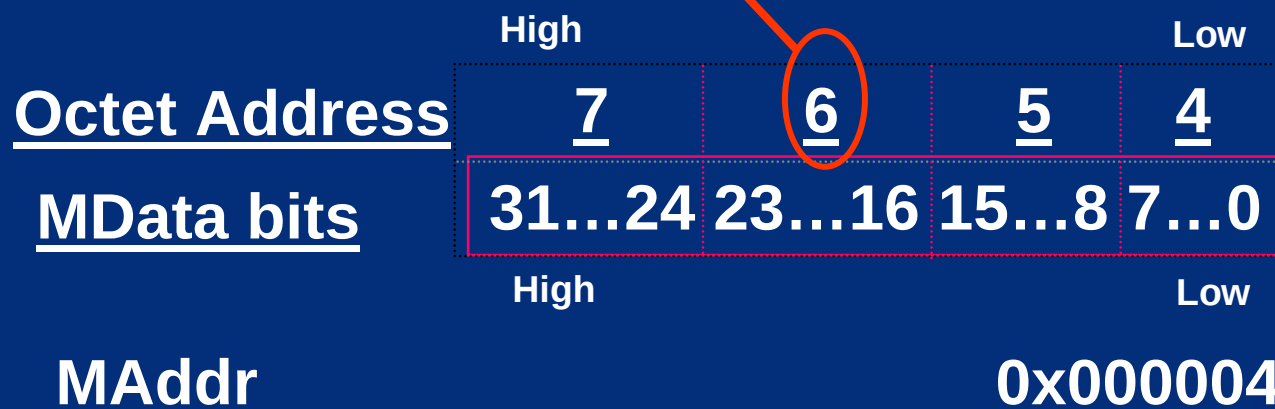
- OCP address space is flat
- Addresses granularity is 8 bits (1 byte)
- Parameter *data_width* indicates the number of bits in a *OCP word*
- Word size is the natural transfer unit of the core, across the OCP socket
- Parameter *addr_width* indicates the number of address bits generated by a core.
 - OCP addresses are byte addresses aligned to words
 - So a core can address $2^{\text{addr_width}} / (\text{data_width}/8)$ OCP words



OCP Address Ranges



Example: Access address(6), bus data_width == 32





Flow Control is optional

- **Accept field can be configured as static**
 - Tie-off value is not configurable
 - Always 1
- **Transfer occurs in 1 cycle, whenever sender initiates it**

Cycle	1	2	3	4	5	6	7	8
MCmd	idle	WR	idle	RD	idle	idle	RD	idle
payload	x	p	x	p	x	x	p	x



Blocking Flow Control

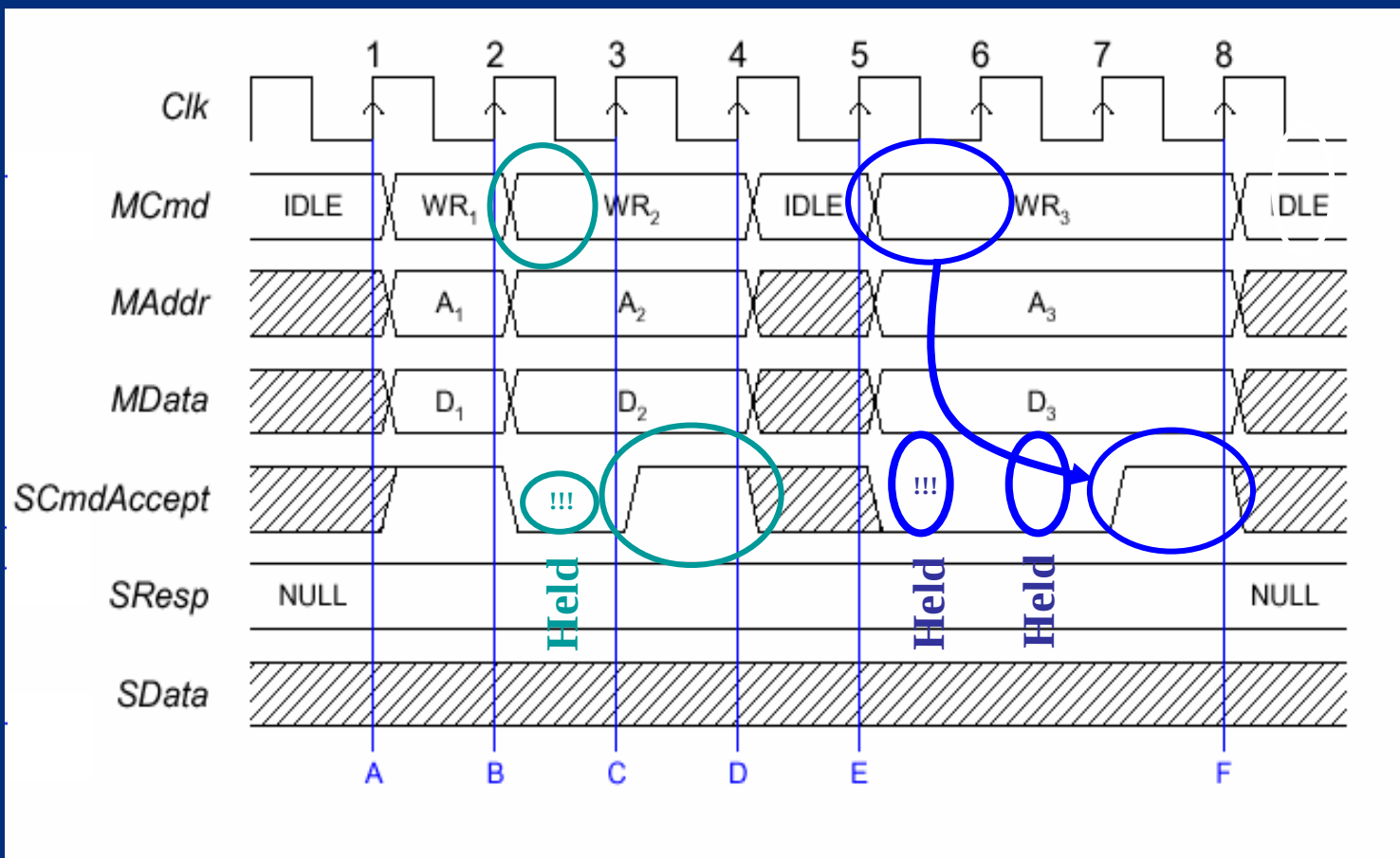
Cycle	1	2	3	4	5	6	7	8
MCmd	idle	WR	WR	WR	RD	idle	RD	RD
payload	x	p1	p1	p1	p2	x	p3	p3
SCmdAccept	x	0	0	1	1	x	0	1

- **Accept field is configured as dynamic**
 - don't care when valid not asserted
- **Phase progresses only when accept asserted**
- **Transfer can occur in any valid cycle**
 - E.g. write occurs in cycle 2, but don't want another command for 2 more cycles

Example Valid-accept flow control: WR



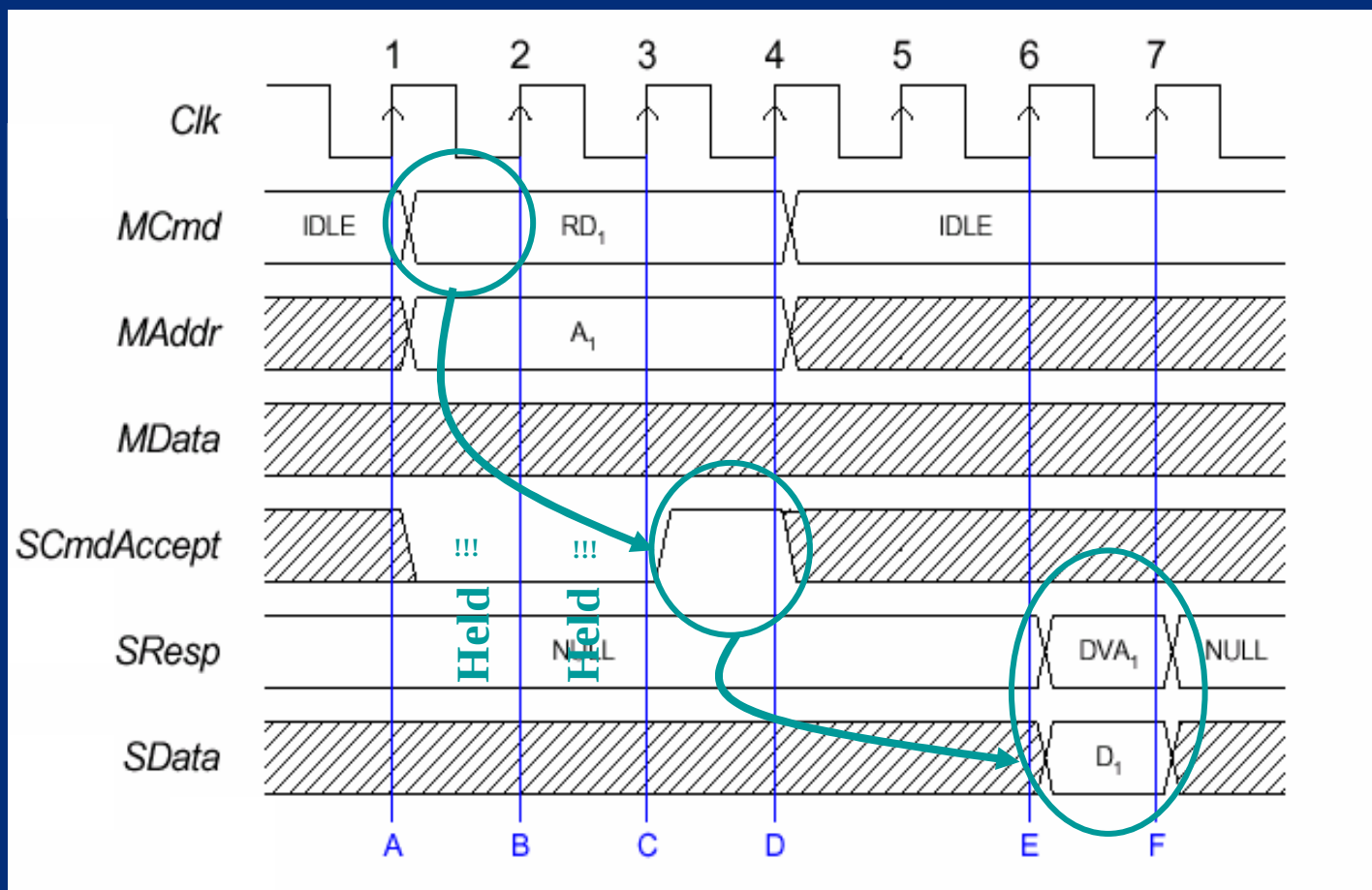
(Delayed Command Accept)



Example Valid-accept flow control : RD



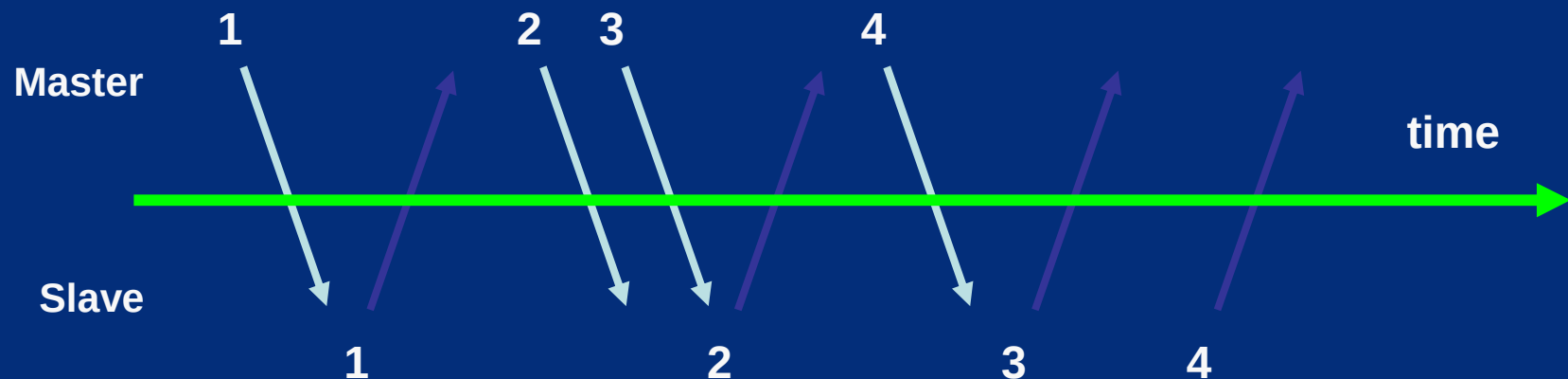
(Delayed Cmd Accept & Decoupled Response)





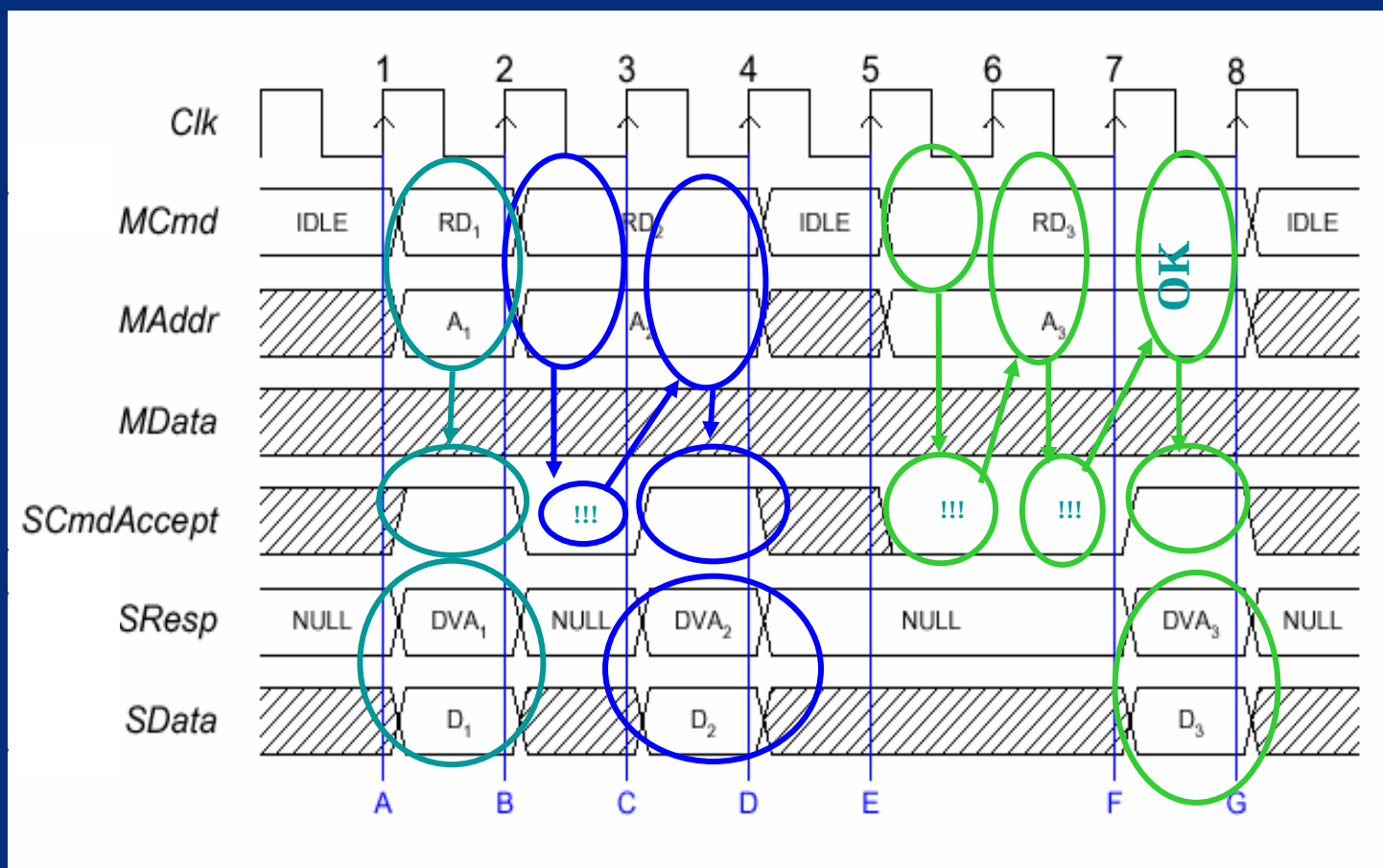
Ordering and Pipelining

- An OCP *transfer* is a complete request/response interaction
- Successive OCP transfers are *strictly ordered* on a thread
- Response return order mirrors the request order
- Successive transfer phases can be pipelined (Multiple requests can be outstanding before the first response returns, assuming the slave accepts the commands)



Covered later, Dataflow signal extensions allow multiple ordered threads which collectively provide *out-of-order* responses.

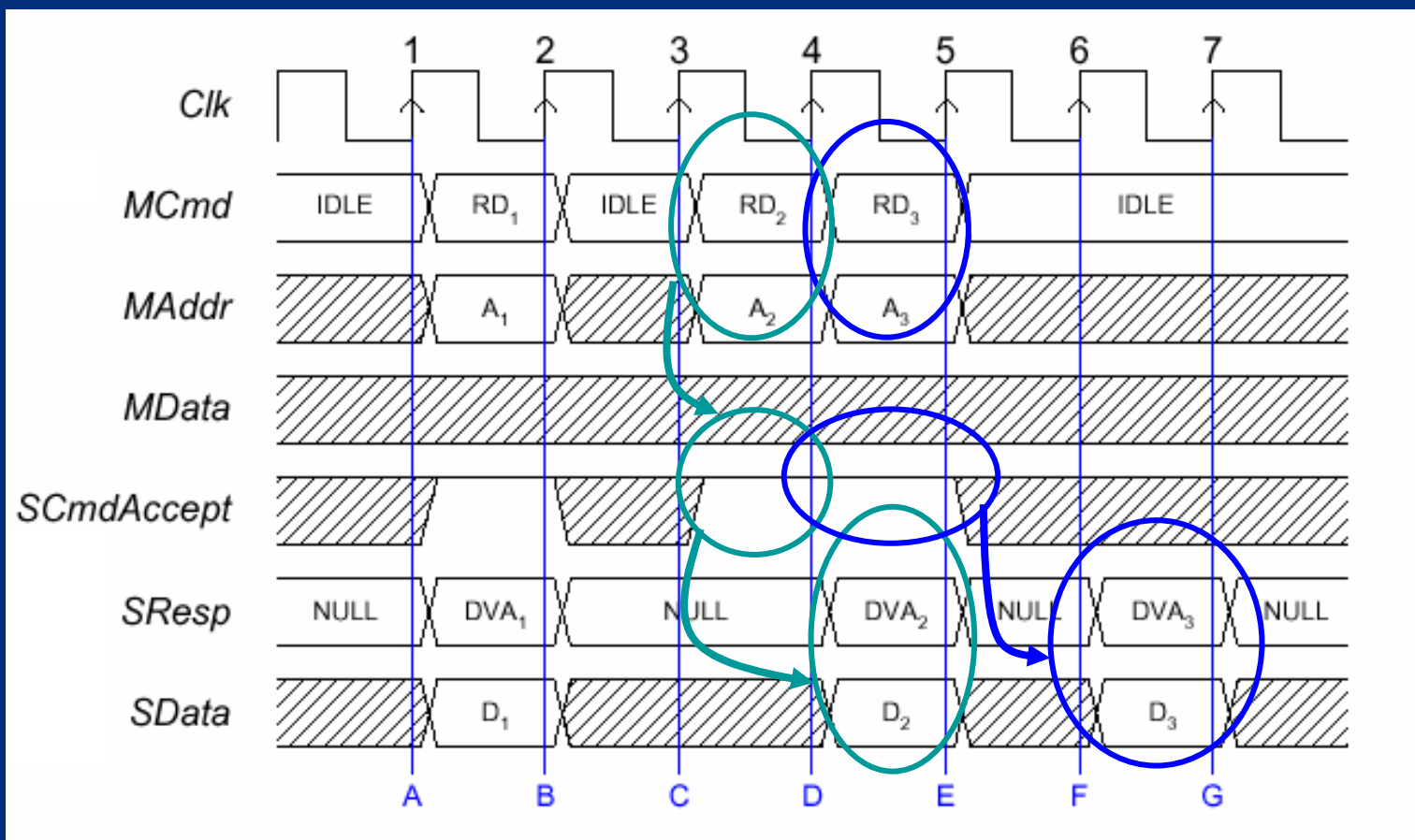
Non-Pipelined Read



Pipelined Read



(Pipelined Read Requests & Responses)





Bursts



OCP-IP2.0 Bursts Details

- **As explained before, bursts are a set of transfers:**
 - Assembled in a transaction
 - Showing a relationship between transfers: the address sequence (locality)
- **With our more detailed view on phases, we can now explain bursts at a finer level of detail**
- **Reminder : 3 categories of bursts are supported**
 - “Imprecise”
 - Speculative length
 - Multiple Request Multiple Data style
 - Early termination legal
 - “Precise”
 - Precise length
 - Multiple Request Multiple Data style
 - No early termination
 - “SRMD”
 - Single Request Multiple Data (packet) style
 - Precise length
 - No early termination



Bursts in OCP-IP2.0

MBurstSeq encodes one of the 7 burst address sequences:

1. Incrementing bursts for block reads/writes - packing
2. Custom – packed
3. Wrapping - packing
4. Custom – not packed
5. Exclusive OR - packing
6. Streaming – non packing
7. Unknown – non packing

MBurstLength encodes the number of transfers in the burst

- Cannot change during the burst if burst is precise
- Is only an hint if burst is imprecise

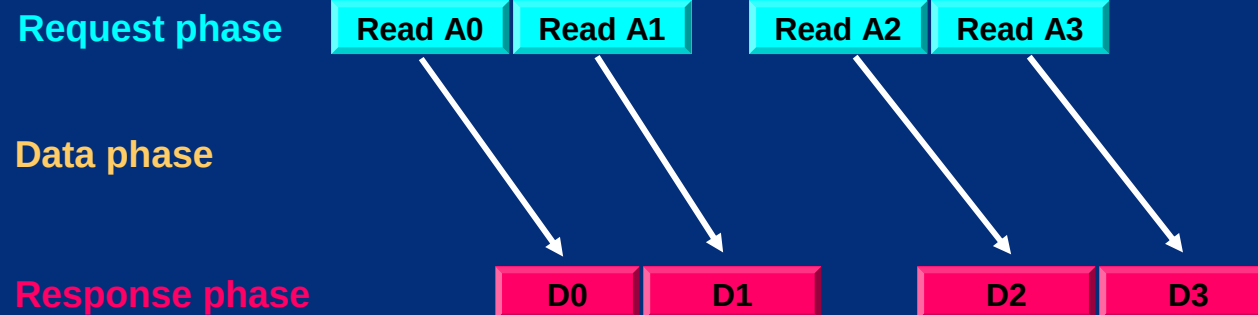


Bursts in OCP-IP2.0 (cont.)

- **Optional framing information**
 - MReqLast, SRespLast indicates the end of a burst
 - But this is optional as :
 - If burst is imprecise, last transfer is indicated by MBurstSize = 1
 - If burst is precise, it is sufficient to count transfers
- **If an initiators can change its burst type dynamically, on a burst by burst basis, additional signals define if the burst is :**
 - MRMD / SRMD
 - Precise / imprecise
- **In case an burst can be chopped into smaller bursts by a target (typically, an interconnect), Atomicity attributes allow control of the chopping process**



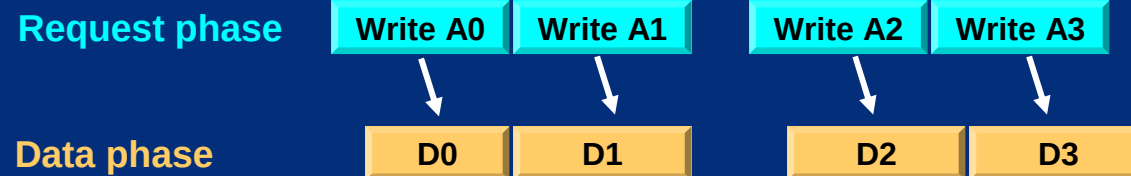
MRMD Read Burst



- Could be imprecise or precise
- Response phase can't begin before Request phase begins
- Response phase can't end before Request phase ends



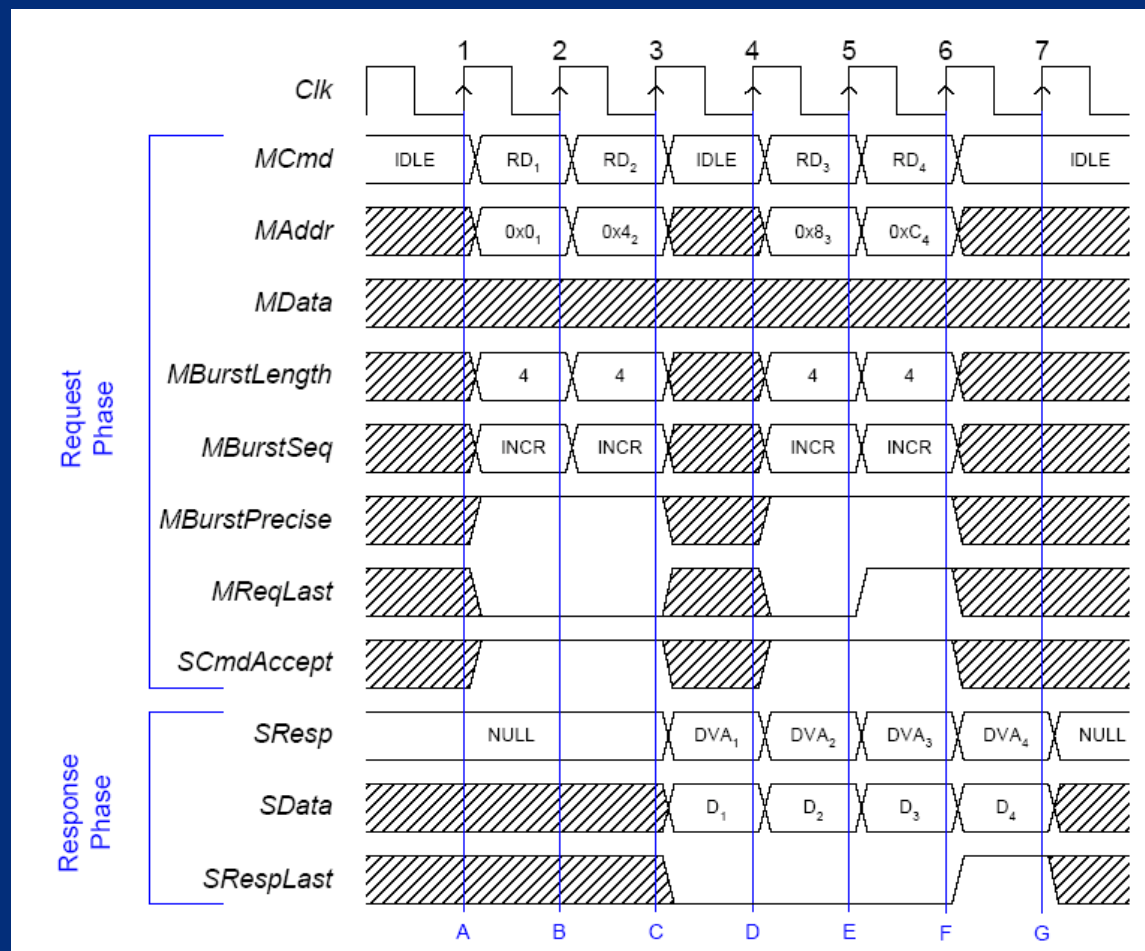
Posted Write MRMD Burst



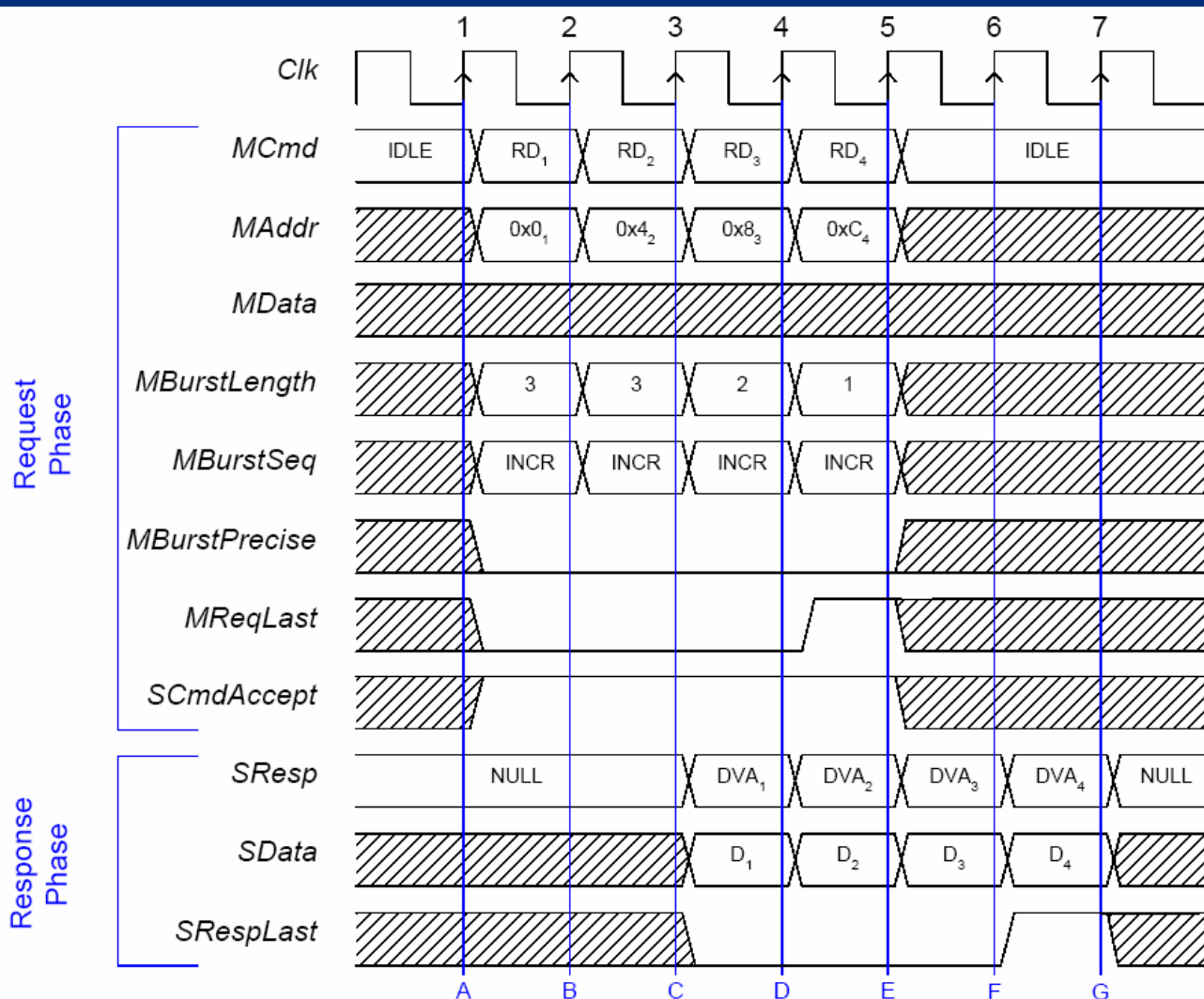
Response phase

- Could be imprecise or precise
- Data phase can't begin before Request phase begins
- Data phase can't end before Request phase ends
- If data phase not present, data payload has same timing as request

OCP-IP2.0 Burst : INCR, Precise, MRMD

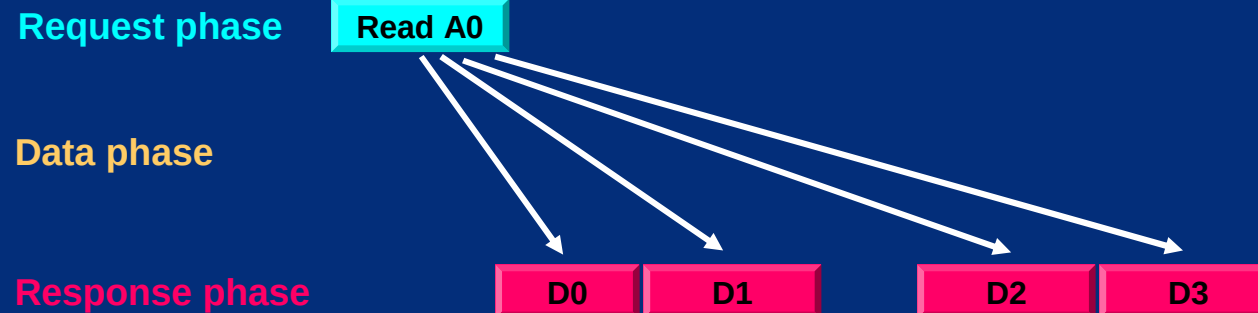


OCP-IP2.0 Burst : INCR, imprecise



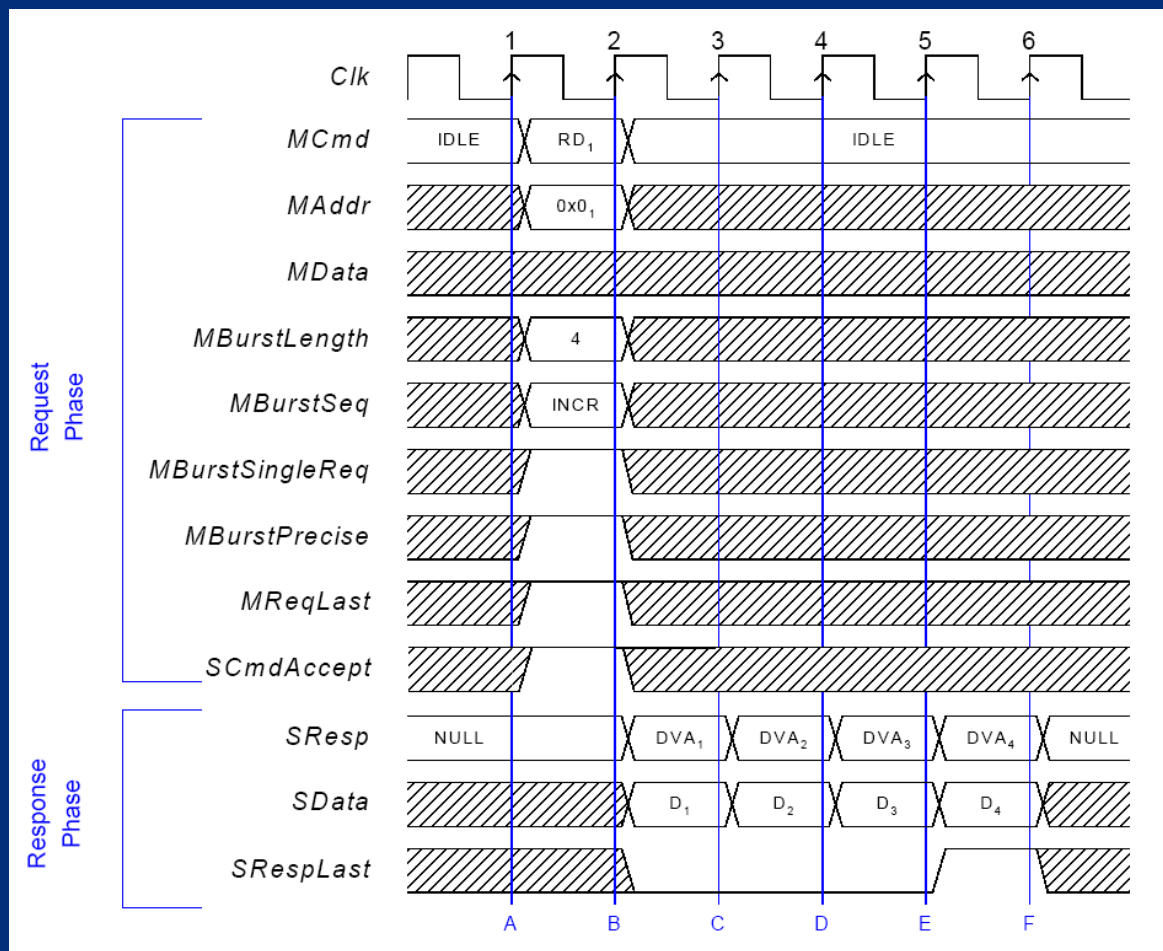


SRMD Read Burst



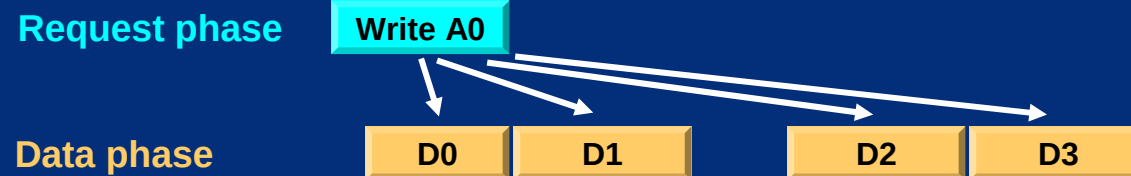
- First Response phase can't begin before Request phase begins
- First Response phase can't end before Request phase ends

OCP-IP2.0 Burst : INCR, Precise, SRMD





Posted Write SRMD Burst



- First data phase can't begin before request phase begins
- First data phase can't end before request phase ends
- Requires Data Handshake to be enabled

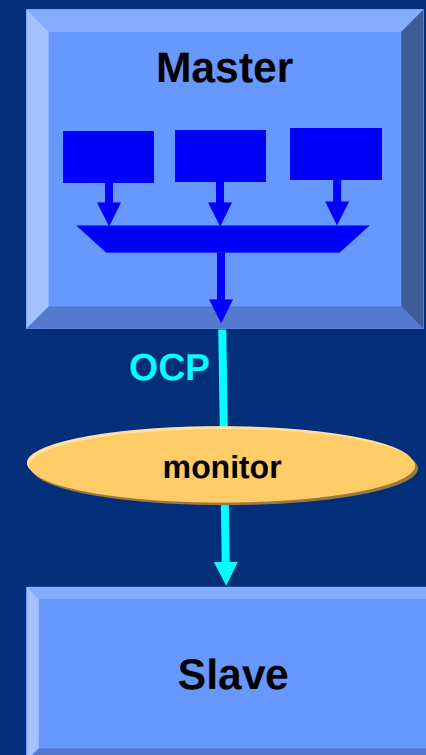


Threads



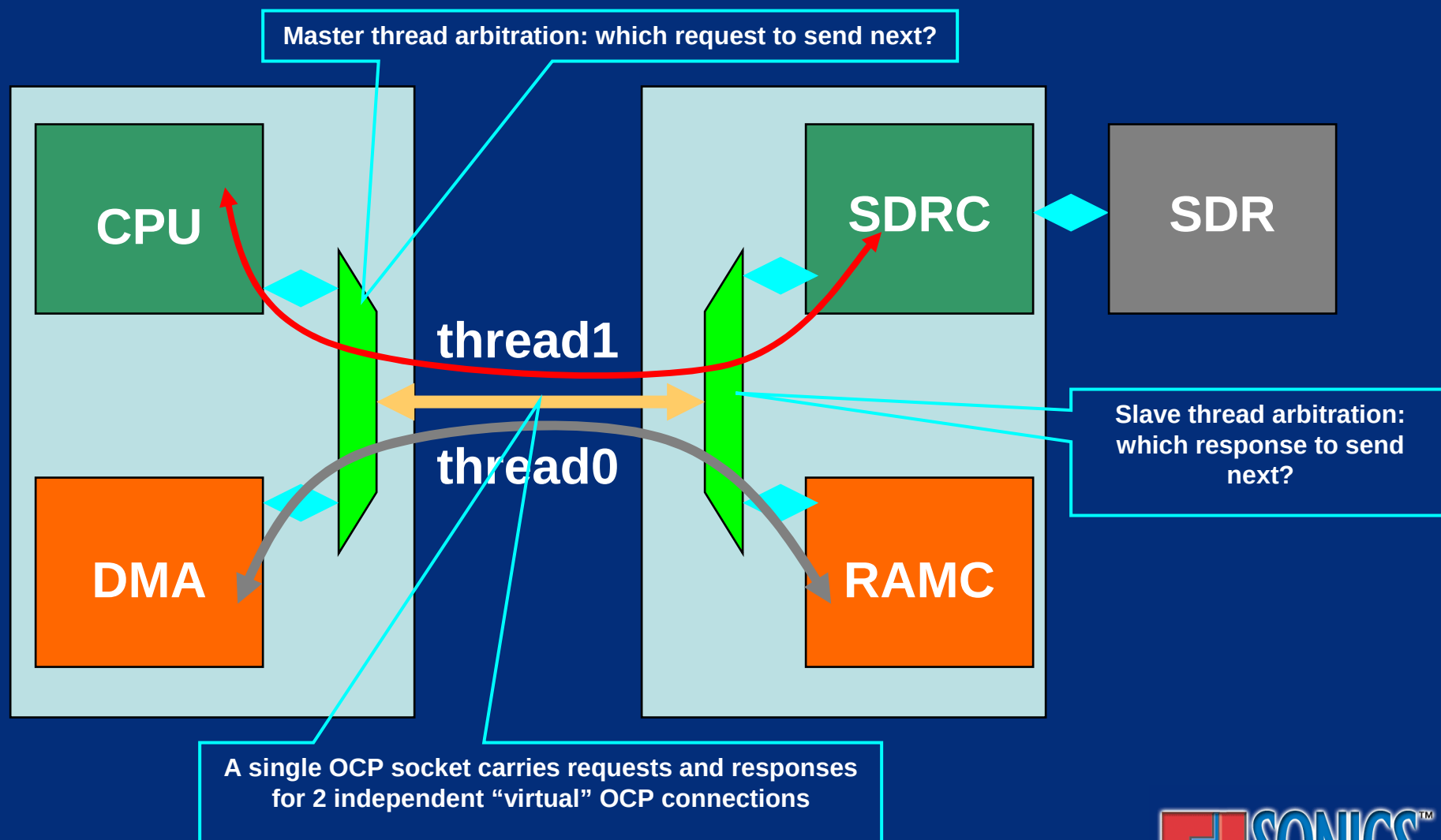
Threads

- **Master can represent a collection of actual physical or logical masters that share a single set of OCP wires**
 - Bridge from bus
 - Sub-system (DSP with DMA)
- **Threads can be seen as multiple logical OCP connections using the same physical socket**
- **Transaction execution order is maintained within each thread (within a logical OCP connection)**
- **Reordering between threads is allowed**



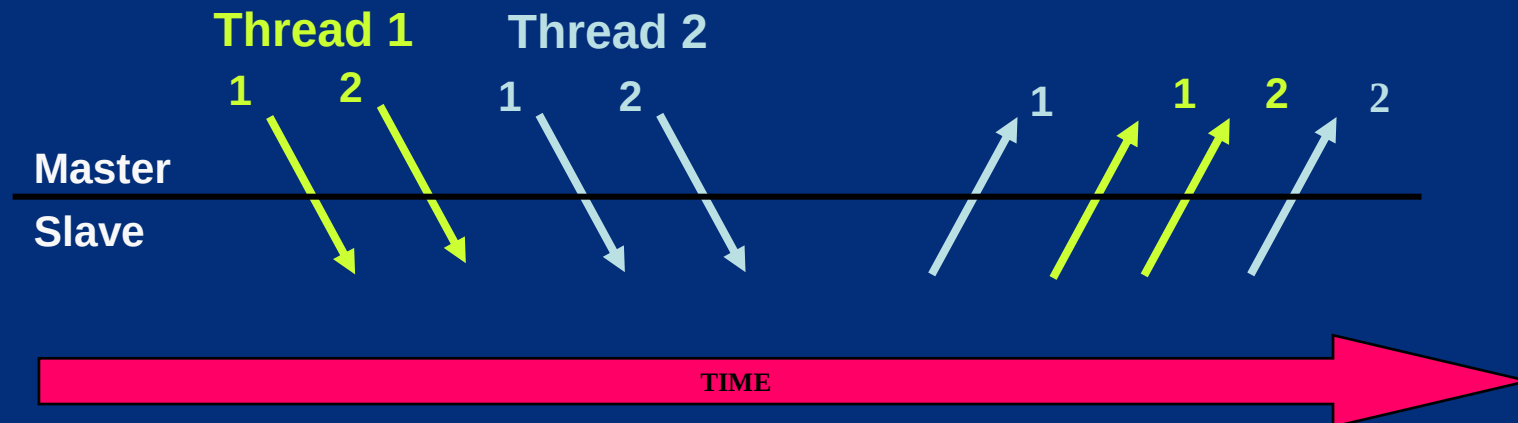


Threads illustrated





Threads and ordering



- Strict ordering within a thread

Thread 1: 1, 2 → 1, 2

Thread 2: 1, 2 → 1, 2

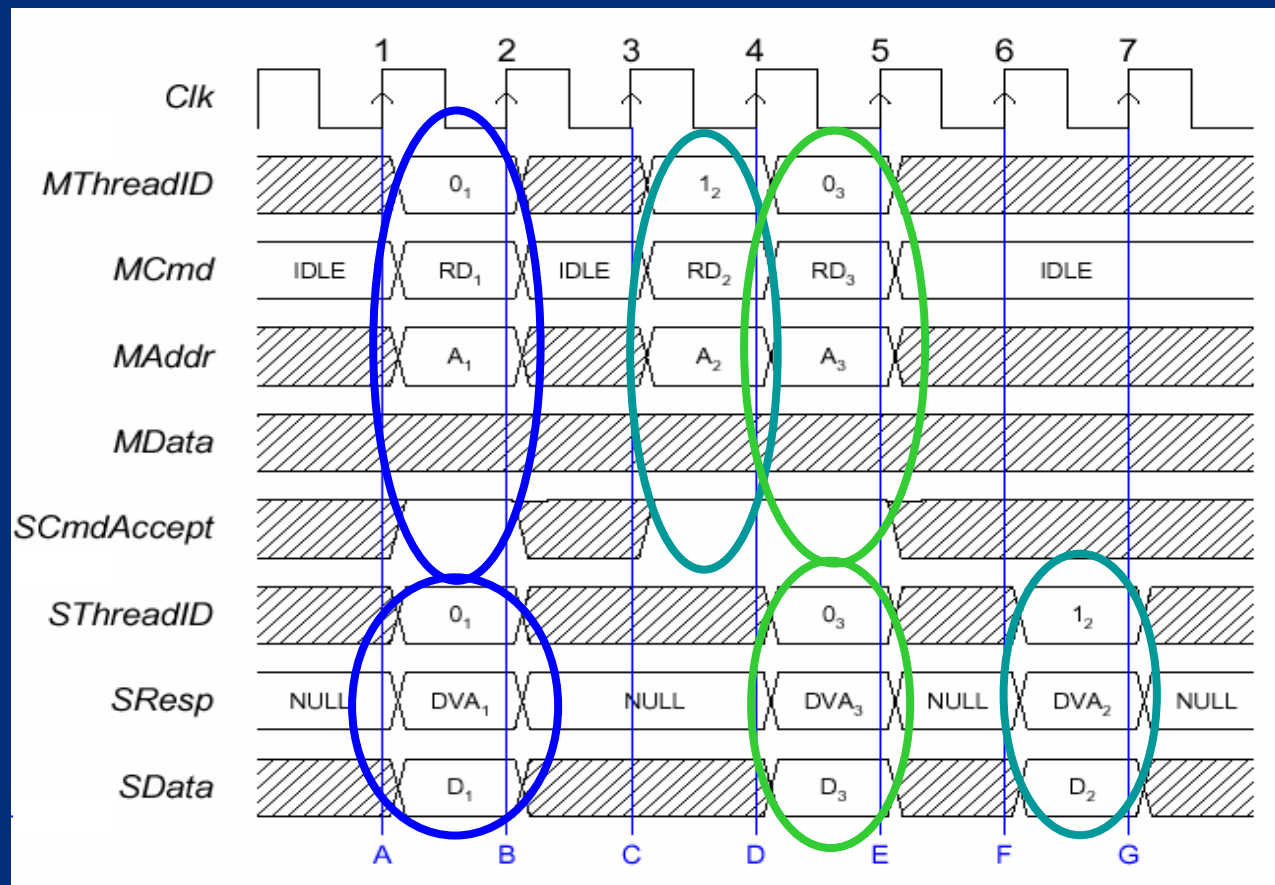
- No ordering between threads

1, 2, 1, 2 → 1, 1, 2, 2

Threaded Read



(Out-Of-Order Threaded Read Completion)





Understanding ThreadBusy

- A core can be busy on one thread, while still able to accept requests for another thread.
- In order to prevent blocking, the slave can use the optional SThreadBusy signals (one bit per thread) to indicate which threads are busy.
 - A Master should not issue requests to busy threads, if those requests will block, but it may do so unless it follows the BusyExact semantics (see below)
- Similar arrangement exists in opposite direction with optional MThreadBusy signals for responses from the slave.
- OCP-IP2.0 offers the BusyExact semantics :
 - If sthread_busy_exact = 1 for a core, the master shall not present any command in a cycle for those threads that are indicated as busy.
 - Idem for mthreadbusy_exact, sdatathreadbusy_exact.



Flow Control view with Threads

Two flow control style: Busy-valid (exact) & Busy-accept (non-exact)

- Each phase can be configured with one of 4 flow control protocols
 - Use of the phase handshake fields
 - Arbitration determines which thread goes next
 - Pacing determines when transfer actually occurs

Flow control protocol	Who arbitrates	Who paces	Phase busy present	Phase valid present	Phase accept present
<i>None</i>	<i>sender</i>	<i>sender</i>	<i>no</i>	<i>yes</i>	<i>no</i>
<i>Blocking</i>	<i>sender</i>	<i>receiver</i>	<i>no</i>	<i>yes</i>	<i>yes</i>
<i>Non-Blocking</i>	<i>receiver</i>	<i>sender</i>	<i>yes</i>	<i>yes</i>	<i>no</i>
<i>Blocking w/ Hints</i>	<i>sender</i>	<i>both</i>	<i>yes</i>	<i>yes</i>	<i>yes</i>



Non-Blocking (exact) Flow Control

Cycle	1	2	3	4	5	6	7	8
SThreadBusy	000	001	111	001	001	000	010	000
MCmd	idle	WR	idle	WR	RD	idle	RD	idle
MThreadID	X	1	X	2	1	X	0	X
payload	x	p1	x	p2	p3	x	p4	x

- **Without retraction, one thread can block another**
 - Receiver knows which threads can make progress
 - Sender does arbitration
 - If sender picks thread that cannot make progress, shared OCP wires are tied up.
- **Busy signals prevent this**
- **Accept signal not needed**
 - Accept is communicated in advance by the busy
- **Called “threadbusy_exact” if both sender and receiver promise to honor busy signals.**

Blocking with Hints (non-exact) Flow Control



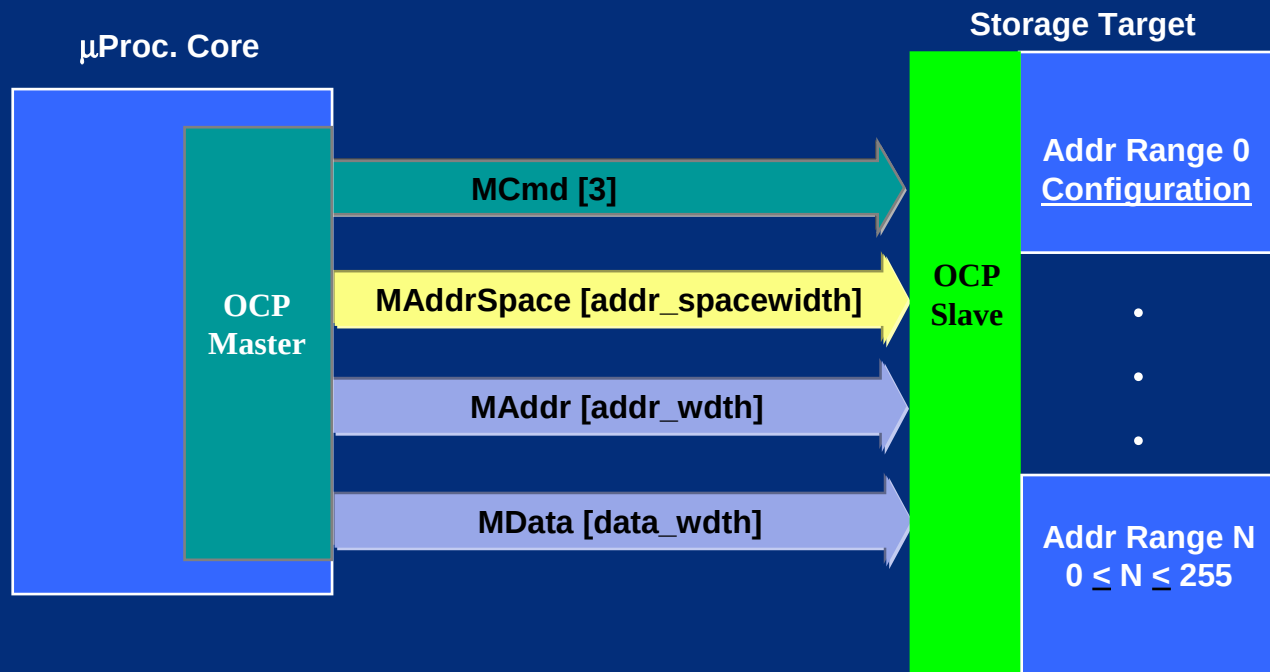
Cycle	1	2	3	4	5	6	7	8
SThreadBusy	000	001	111	001	000	001	001	000
MCmd	idle	WR	idle	WR	RD	RD	RD	idle
payload	x	p1	x	p2	p3	p3	p4	x
MThreadID	x	1	x	2	0	0	0	x
accept	x	1	x	1	0	1	1	x

- Either sender or receiver doesn't always honor busy signals.
 - Cycle 5
- Accept signal configured as dynamic
- Some inter-thread blocking occurs



Other Protocol Features

Alternate address space





Byte Enables

- **The basic OCP data transfer size is a *word***
 - Corresponds to the OCP parameter *data_width*.
- **When using the byte enable extension, the interface provides one signal per byte to indicate the presence of a valid data byte.**
 - Allows for smaller transfers than the OCP word size
 - Can assist with endian matching
- **Any combination of byte enables is legal (*general byte enables*)**
 - example: can have byte enable 0x35 on an 8-byte OCP
- **BUT: can be restricted to combinations of byte enables which are aligned to powers-of-two (*force aligned*)**
 - some cores cannot handle general byte enables

Request and data qualifiers (in-band extension)



- **MReqInfo**: OCP allows requests to be qualified by additional, user-defined, data that will be routed along with the command
 - Implementation of requests attributes like secure mode, cacheable, debug, ...
 - Supported by both OCP-S2.4 and OCP-IP2.0
- **SRespInfo**: same purpose as MReqInfo, but for responses. OCP-IP2.0 only.
- **MDataInfo / SDataInfo** : OCP-IP2.0 allows also data to be qualified, independently of the request.
 - Allows 2 kinds of info to be transmitted with the data: a per-byte field, and a per-word field
 - Typical use : parity



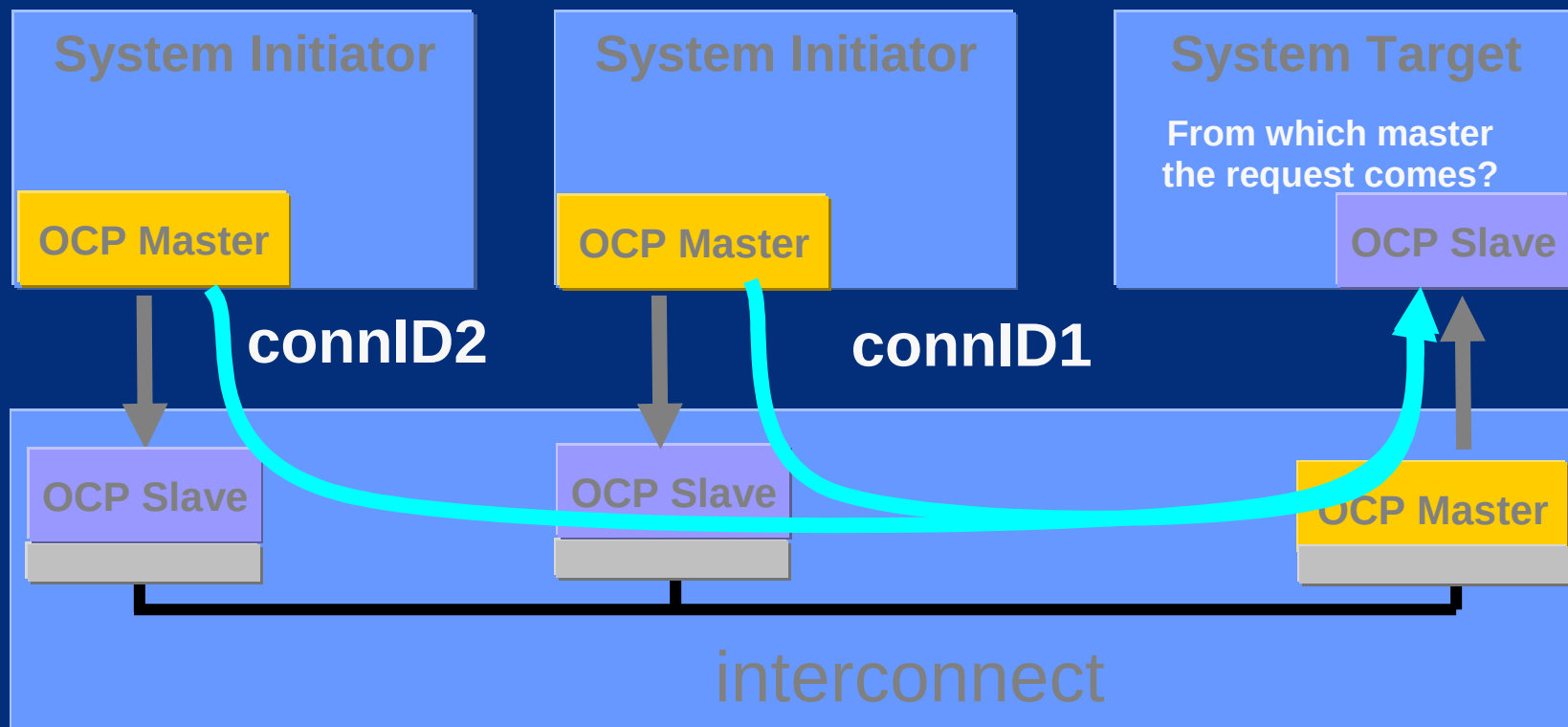
Connections

- **Connection (MConnID) is a tag that travels with a request :**
 - It has nothing to do with threads! It does not change requests / response order.
 - It is given to the initiator in a user-defined fashion (no rules enforced by the standard)
 - It shall be preserved by the interconnect from initiator to target
 - It helps the target and/or the interconnect to know from which initiator a request is coming from
- **This knowledge is use typically for Quality of Service purposes**
 - **in case of conflicting access to a shared resource**

Connections across interconnects



Connection information is preserved across the interconnect





Sideband Signals



Sideband Signals

- **Generation and delivery synchronous to OCP clock**
- **Transit asynchronous to data flow**
- **Not for analog signaling**
- **Signal Names:**
 - Reset
 - Interrupt
 - Error
 - Core Flags
 - Control/Status

Reset



- **Reset is synchronous to the OCP clock**
- **This is NOT the reset of the core but of the socket**
 - Must be seen as a protocol signal, used to restart transactions from scratch. Clear transaction history.
- **Must be asserted for at least 16 cycles**
- **OCP-S2.4 Reset :**
 - Is optional
 - Is sourced by the master, the slave or the system
- **OCP-IP2.0 Reset :**
 - Is **mandatory!**
 - Can be sourced by the master (MReset_n)
 - OR can be sourced by the slave (SReset_n)
 - OR be by sourced by both master and slave



Core Flags

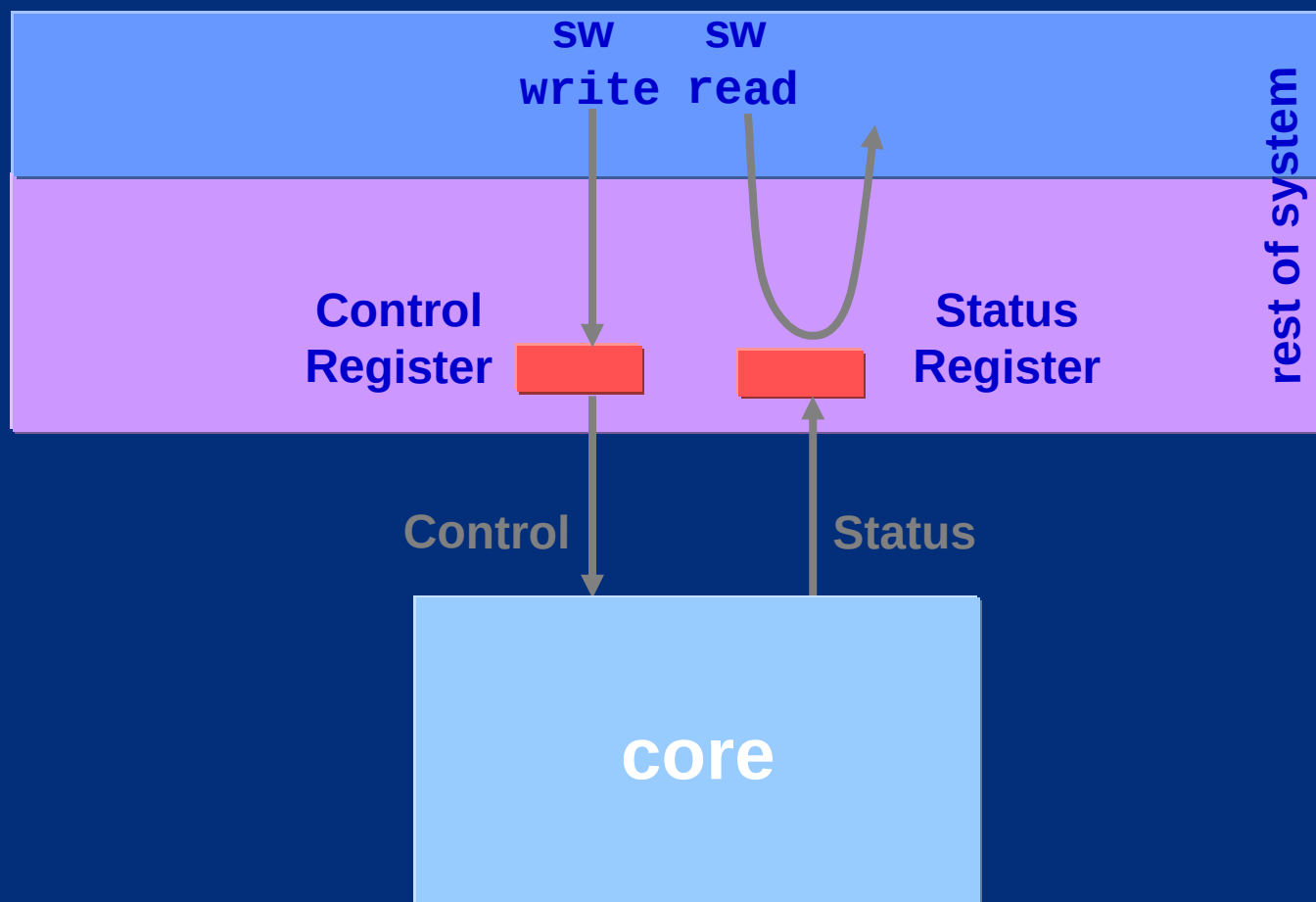
- **Master and Slave flags (MFlags and SFlags) are generic, out-of-band signals driven by master and slave respectively**
 - Examples
 - High-level flow control
 - Multiple interrupts
 - Handshaking
 - Narrow data links
- **OCP puts no restriction on their use**
- **No analog signals allowed though**
- **Typically for hardware-to-hardware interactions**



Control and Status

- **Often, cores have almost static control and status information on their interfaces.**
 - Control - mode bits
 - Status - indication that an operation is complete
- **OCP configurations can include control and status signals as well as event and busy notification**
 - New control value is presented
 - Status is changing
- **Major difference to core flags: control/status is typically wired through a register and interacts with system software**

Control and Status (2)





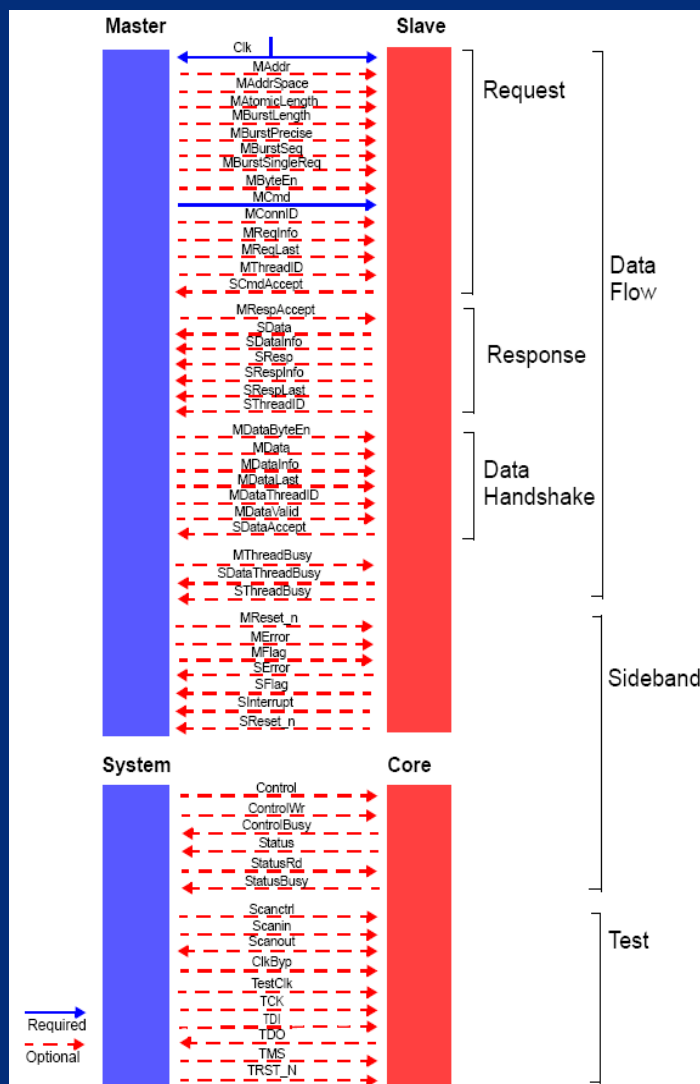
Test Signals



Test Ports

- **Mainly for hard macros...**
- **Scan**
 - With one or more chains
- **IEEE 1149 (JTAG)**
 - Standard JTAG interface
- **Clock control**
 - Used to supply a different clock for chip test

Summary : OCP sockets signals





Summary (2) OCP Socket Signals

1. Dataflow Signals

- Associated with moving data in and out of cores
- Includes address, data, handshaking, commands, etc.
- Similar to traditional computer bus services
- *Basic* Dataflow signals are required
- Optional Dataflow signals
 - *Simple* Extensions
 - *Complex* Extensions (OCP-S2.4)
 - Burst Extensions (OCP-IP2.0)
 - Thread Extensions (OCP-IP2.0)

2. Sideband

- Associated with non-data communication
- Not synchronized to data flow (out of band)
- Examples: interrupts, DMA requests, high-level flow control, etc.

3. Test (Optional)

- Related to debug and manufacturing test



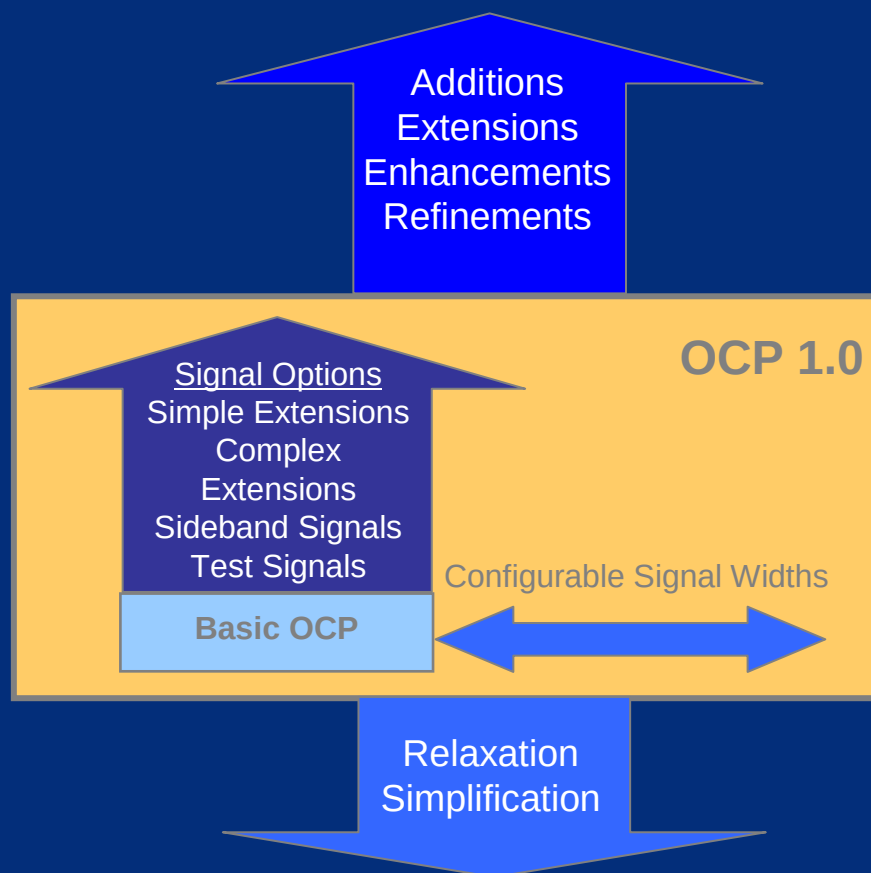
OCP 1.0 to 2.0 Changes

Highlighting what is different

OCP 2.0 Evolution



OCP 2.0 Specification Standard of Record



- **Evolutionary**
- **Non-disruptive**
- **Leverages 1.0 skills and training**
- **Extends OCP applicability**
- **Increases value of OCP skills**
- **Enhanced by users**
- **Addresses current & future needs**
- **Functional superset of OCP 1.0**



OCP 2.0 Changes

- Burst Model is substantially revised
 - Critical word first, imprecise-length and exact-length bursts
 - Single request bursts that transfer multiple data
 - Burst framing information to accompany all request transfer phases
- Write Response
 - Provides means to indicate whether a write is posted or not
 - Separates posting semantics concepts from write response concepts



OCP 2.0 Changes (cont.)

- **Endianness**
 - Provides endianness information on an OCP interface
 - Allows endian-neutral interconnects to become endian-adaptive when the interconnect must choose an endianness
- **System-Level Design Protocol Subset**
 - Relaxes Basic OCP signal requirements to simplify interfaces for simple IP blocks requiring unidirectional, non-addressed or odd data size capabilities, etc.



OCP 2.0 Changes (cont.)

- **In-band Signaling**
 - Supports designer extensions in-band with all phases
 - Can be used for data tagging (e.g. byte parity, ECC)
 - Data tags can associate with data words or specific data bytes
- **Dual Reset**
 - Adds a second, optional, OCP reset signal so each OCP Master and Slave can have both a reset output and input
 - Either (or both) side can reset the interface



OCP 2.0 Changes (cont.)

- **Exact ThreadBusy**
 - Ensures non-blocking multi-threaded OCP interfaces
 - Allows cores to indicate they observe precise ThreadBusy semantics, guaranteeing multi-threaded OCP interface can be non-blocking
- **Lazy Synchronization**
 - Provides OCP with necessary signals to ensure *read-linked / write-conditional* semaphore handling support
 - Permits simultaneous support with *read-modify-write* semaphore handling for processors that require it



Implementation

Practical considerations



About timing



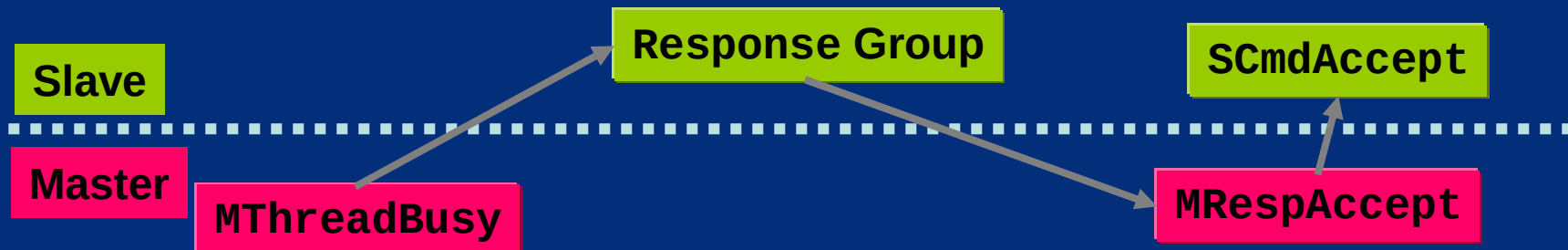
Timing Guidelines

- **OCP protocol defines legal combinational paths**
 - Legal combinational dependencies are only within the data flow signals
- **OCP protocol defines 3 Levels of timing**
 - See them as timing budgeting guidelines
 - These are provided to give information to system level designers and integrators



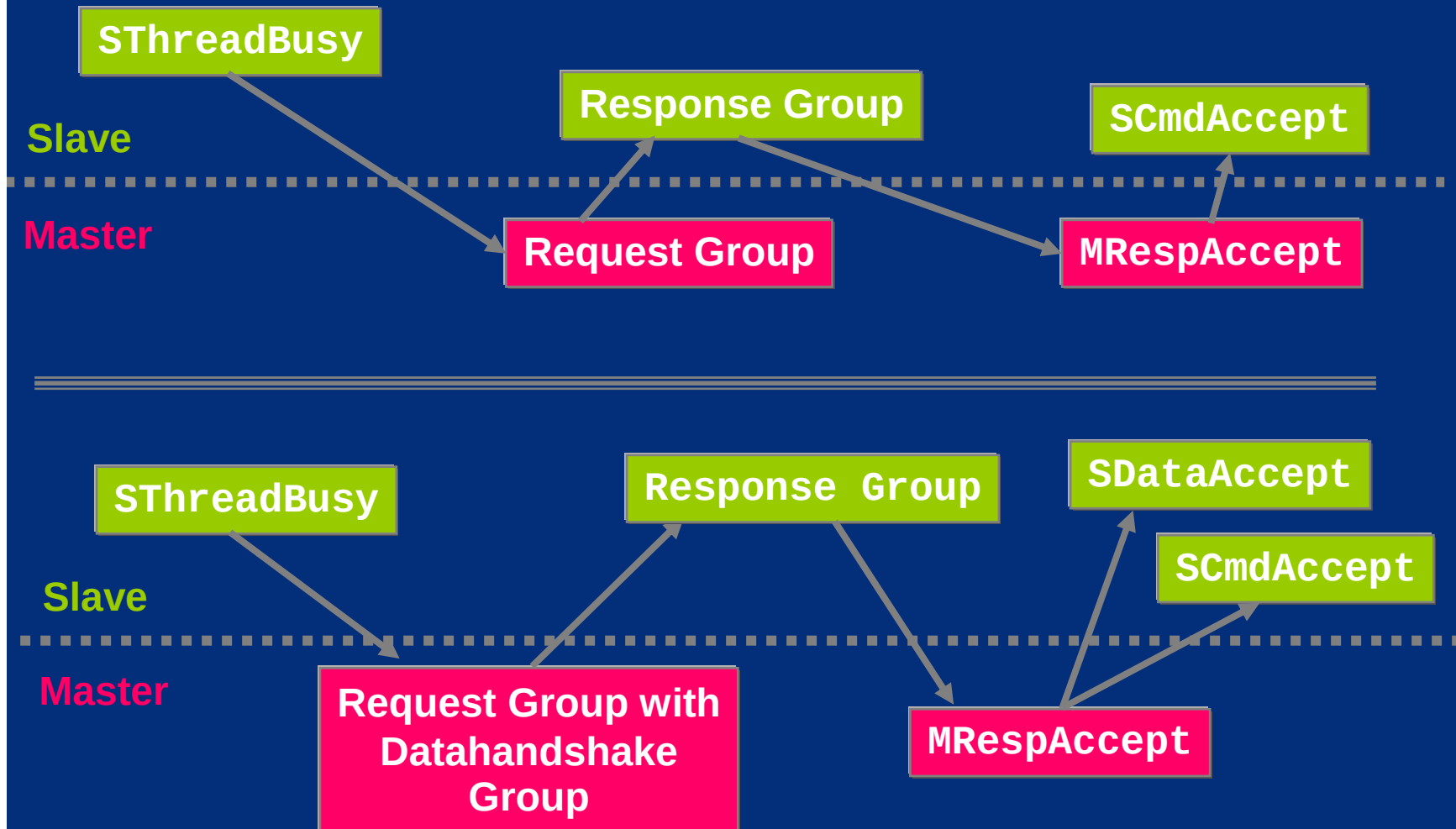
Combinatorial Dependencies

- To ensure that all timing arcs have a beginning and an end, only certain *combinational dependencies* are allowed
 - Restrictions simplify static timing analysis
 - In a combinational dependency, an output signal is dependent upon the current value of an input signal *in the same OCP clock cycle*
 - The complete set of allowed dependencies are too complex (i.e. slow) for most cores - use some registers!





Combinatorial Dependencies(2)





OCP timing levels

- Describes core interface timing at a very high level
 - Only applies to dataflow and sideband signals
 - No timing guidelines for the scan and test related signals
 - OCP protocol defines a syntax for core timing description
-
- Level0: core interface design does not adhere to any specific timing
 - Level1: core interface design has conservative interface timing
 - Level2: core interface design has high performance interface timing

OCP timing levels(2)



Interoperability Between Levels

	<u>Level0</u>	<u>Level1</u>	<u>Level2</u>
Level0	X	X	X
Level1	X	V	V
Level2	X	V	V*

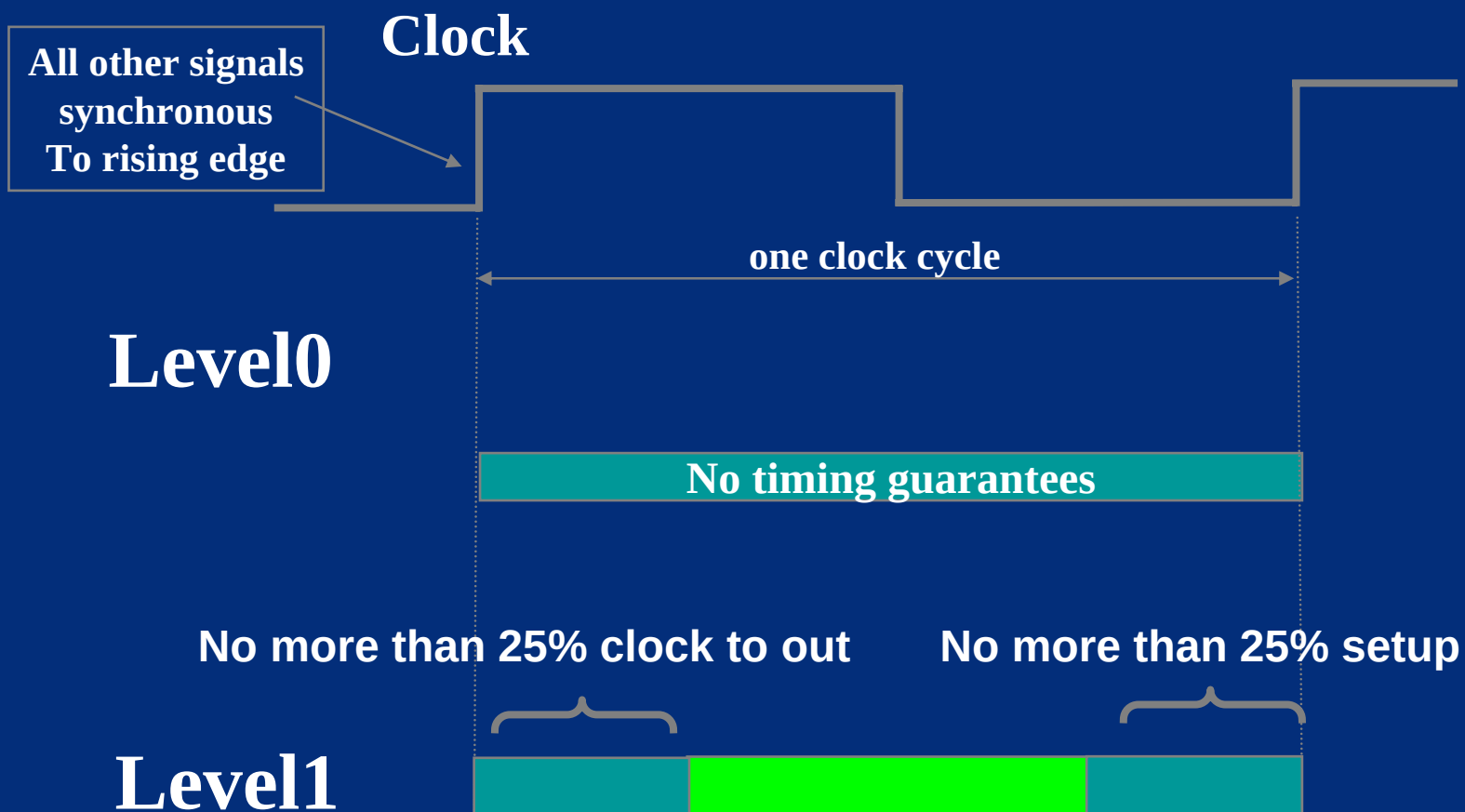
X: no guarantee

V: guaranteed interoperability with possible performance loss (extra latency)

V*: high performance interoperability with possibly minor changes



OCP timing budgeting : level0 and 1





Level 2 timing budget per signal

Signals	Slave		Master	
	Output % cycle	Setup % cycle	Output % cycle	Setup % cycle
Control, Status	25	75	25	75
ControlBusy, StatusBusy	10	90	10	90
ControlWr, StatusRd	25	75	25	75
Datahandshake Group (MdataValid, MDataThreadID Mdata (if Datahandshake=1))		70/40	30/60	
MDataThreadID		-/50	-/50	
MRespAccept		50/25	50/75	
MThreadBusy		-/90	-/10	
MThreadID		-/50	-/50	



Level 2 timing budget per signal(2)

Request Group (Maddr, MaddrSpace, Mburst, MbyteEn, Mcmd, MconnID Mdata (if Datahandshake = 0))		70/40	30/60	
Reset_n		90	10	
Response (SData,SResp)	30/60			70/40
SCmdAccept	50/75			50/25
SDataAccept	50/75			50/25
SError, SFlag, SInterrupt	40			60
MFlag		60	40	
SThreadBusy	-/10			-/90
SThreadID	-/50			-/50

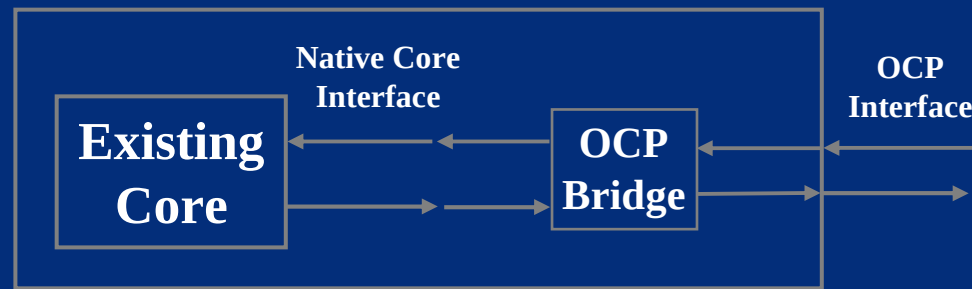


Adding an OCP interface to a core: Conversion, wrapping



Core Conversion Methodology

Converting logic blocks to packaged cores



The Process

1. Know the native core interface
2. Know OCP
3. Build Bridge
4. Test
5. Package the OCP core



Typical Conversion Flow

- Determine the OCP characteristics that the core will have
- Design conversion logic to *wrap* the core
- Describe the core's interfaces and timing constraints
- Convert (or develop) a portable test bench for the core
 - Also tests the new wrapper logic for OCP compliance
- If the core is synthesizable, develop a technology-independent synthesis script for it (optional)
- Assemble core, models, test bench, documentation, timing constraints, synthesis scripts, etc. and package



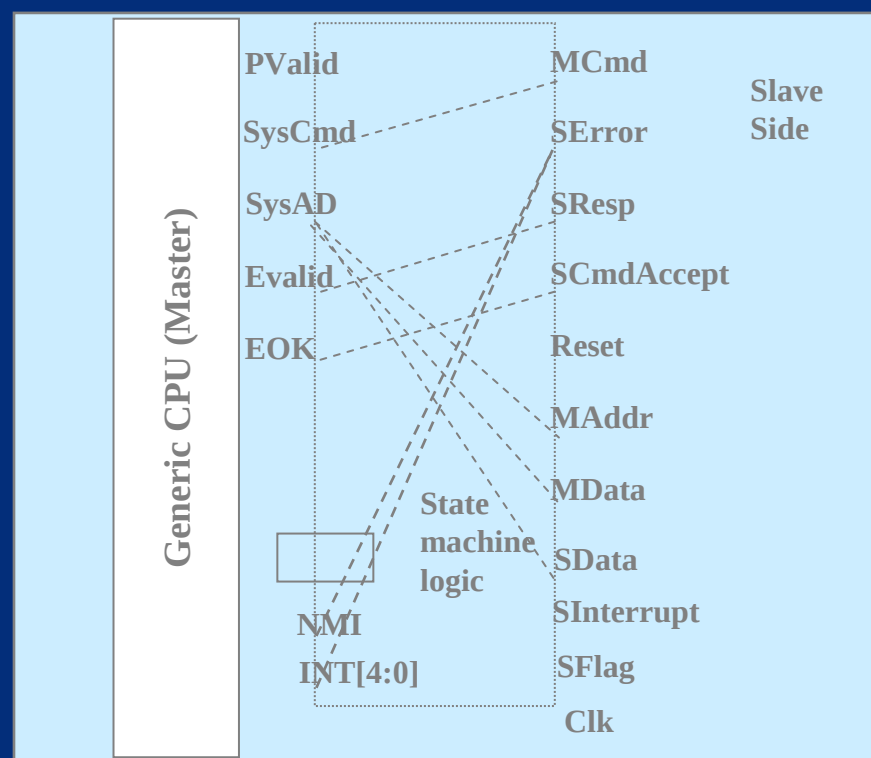
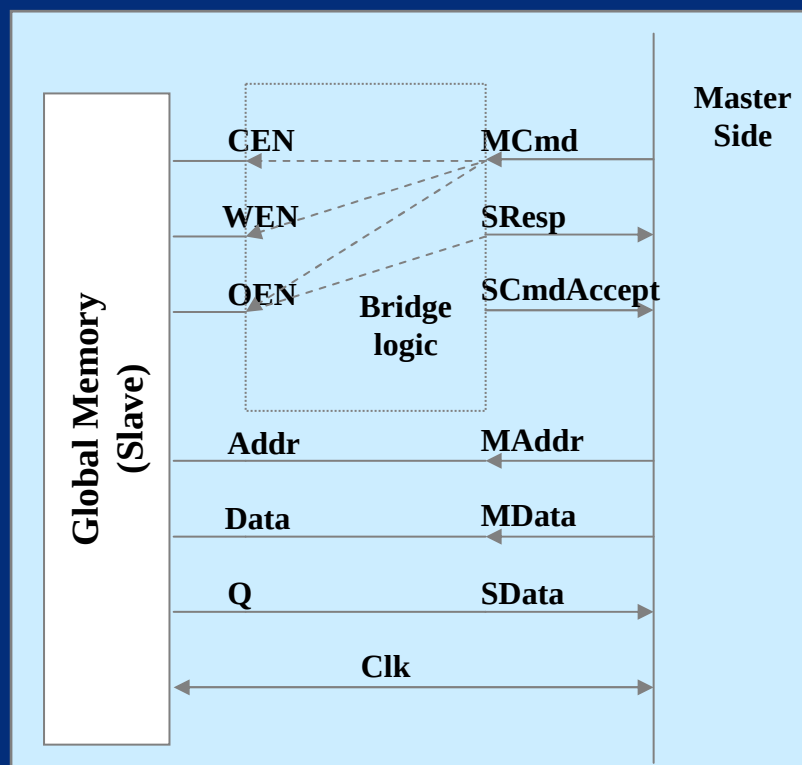
Effective Handshaking

- Use the different OCP handshaking signals to reduce interface logic storage needs.
- If a given Accept signal is not asserted, the sender of that signal must hold the corresponding field invariant until the receiver is ready.
- **Example**
 - Target core is sequencing the command address over a narrow internal bus
 - Leave SCmdAccept de-asserted until all of the address has been sequenced
 - No new storage to hold portions of the address is required

Protocol Patterns



Synchronous SRAM vs. Pipelined Processor Bus





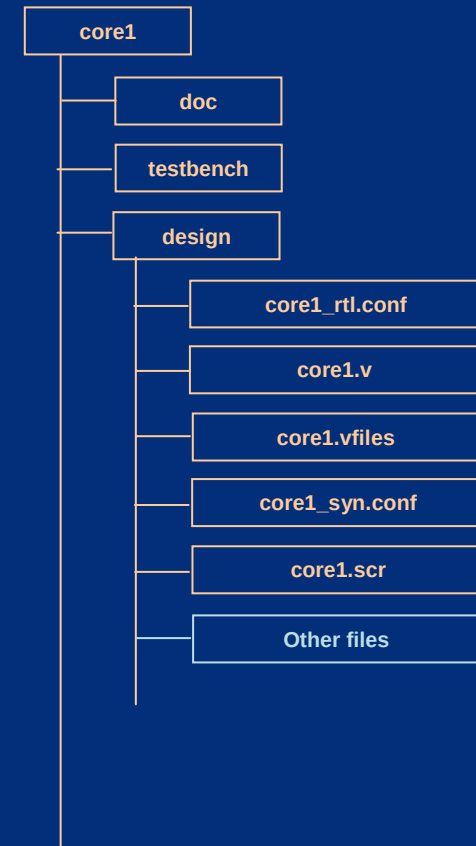
Usually Relevant OCP Signals

	CPU Bridge	Bus Bridge	Flash Ctrl	Memory Ctrl	DSP
MBurst	X	X		X	X
MByteEn	X	X	X	X	X
MDataValid		X		X	X
MRespAccept	X				
SDataAccept		X		X	
M/S ThreadID		X		X	X
MDataThreadID		X		X	X
M/S ThreadBusy	X			X	X
Reset_n	Recommended for all OCPs				
Sinterrupt	X	X			
Serror	X	X	X	X	X
M/S Flag	X			X	X
Control		X			X
Status					X



OCP-IP Core Directory Structure

- **Top level directory has name of the core**
- **May have other sub-directories**
 - Doc, testbench
- **Must have design sub-directory**
 - Contains core_rtl.conf (core description and configuration)
 - Contains core_syn.conf (core interface timing information)
 - Contains top level verilog or vhd file named core.v or core.vhd
 - May contain other Verilog or VHDL files, and a file include list (core.vfiles)
 - May contain core.scr (synthesis script file)





Core Libraries

- **Usually want to package a set of cores**
 - Functionality bundle (Qmodels)
 - Cores for a design
- **OCP-IP standard way of doing it**
 - Makefile if needed

